TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

1. Datos informativos

Asignatura:	Teoría de la programación
Docente:	José Oswaldo Guamán Quinche
Estudiante:	Alison Nagua
Curso / Paralelo:	Ciclo 1 "A"
Fecha de entrega:	03/11/2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Estructura del trabajo (puntos a desarrollar por cada algoritmo)

EJERCICIOS DIFICILES

3.1. Enunciado del algoritmo

Ejercicio 191:

Diseñe un algoritmo que valide una contraseña ingresada por teclado (cadena). Use constantes simbólicas MIN_LONGITUD = 8 y MAX_LONGITUD = 20. Evalúe la expresión lógica compuesta: (longitud(contraseña) >= MIN_LONGITUD AND longitud(contraseña) <= MAX_LONGITUD AND contiene_mayuscula(contraseña) AND contiene_digito(contraseña)). Asigne el resultado a una variable booleana es_valida. Muestre por pantalla mensajes informativos indicando qué criterios se cumplieron y cuáles no. Identifique claramente cada tipo de dato simple utilizado y justifique su elección.

3.2. Análisis del problema

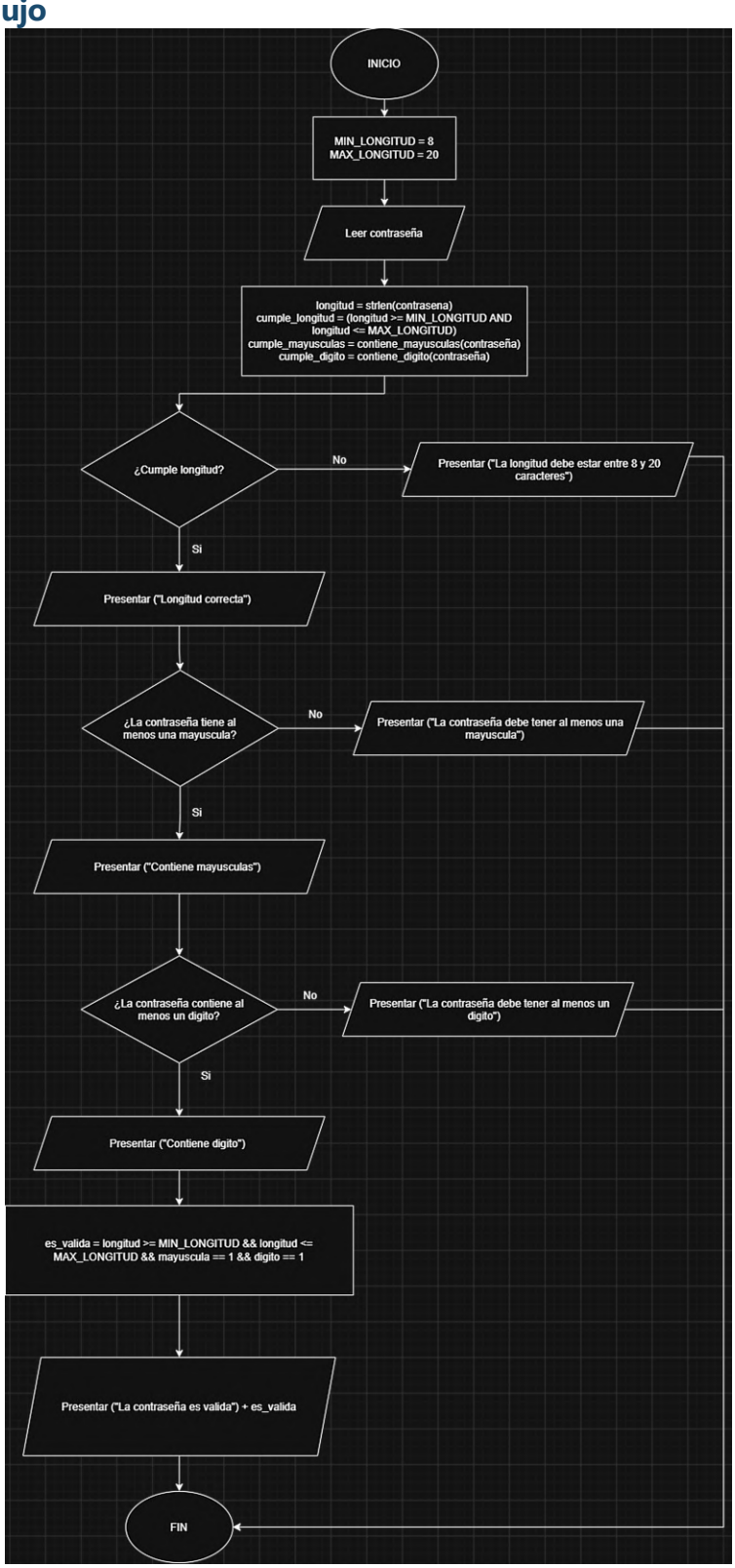
Datos de entrada

1. La contraseña

Datos de salida

1. Mensaje indicando si la longitud es correcta
2. Mensaje indicando si contiene mayúscula
3. Mensaje indicando si contiene digito
4. Mensaje indicando si la contraseña es valida

3.3. Diagrama de flujo



3.4. Pseudocódigo

```
INICIO
//Declarar variables
MIN_LONGITUD = 8 como constante;
MAX_LONGITUD = 20 como constante;
Leer contrasena como cadena;
//Entrada
contrasena[20];
//Proceso
longitud = strlen(contrasena);
cumple_longitud = (longitud >= MIN_LONGITUD AND longitud <= MAX_LONGITUD);
cumple_mayusculas = contiene_mayusculas(contraseña);
cumple_digito = contiene_digito(contraseña);
Si cumple_longitud
//Salida
Presentar ("Longitud correcta");
Sino
Presentar ("La longitud debe estar entre 8 y 20 caracteres");
FinSi
Si cumple_mayusculas
//Salida
Presentar ("Contiene mayusculas");
Presentar ("La contrasena debe tener al menos una mayúscula.");
FinSi
Si cumple_digito
//Salida
Presentar ("Contiene digito");
Presentar ("La contrasena debe tener al menos un dígito.");
FinSi
es_valida = longitud >= MIN_LONGITUD && longitud <= MAX_LONGITUD && mayuscula == 1 && digito ==
1;
Si es_valida = Verdadero Entonces
//Salida
Presentar ("La contrasena es válida.");
Sino
Presentar ("La contrasena no es válida.");
FinSi
FIN
```

3.5. Prueba de escritorio

Prueba	Contraseña ingresada	Longitud	Mayúscula	Dígito	Resultado
1	Alison	La longitud debe estar entre 8 y 20 caracteres	Contiene mayúscula	La contraseña debe contener al menos un digito	Contraseña no válida
2	Alison1234	Longitud correcta	Contiene mayúscula	Contiene digito	Contraseña válida
3	Alisonnaguacruz	Longitud correcta	Contiene mayúscula	La contraseña debe contener al menos un digito	Contraseña no válida

3.6. Codificación en C

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
char contrasena[21];
int MIN_LONGITUD = 8, MAX_LONGITUD = 20;
int longitud = 0, mayuscula = 0, digito = 0;
int es_valida;
    printf("Ingrese contrasena: ");
    scanf("%22s", contrasena);
    longitud = strlen(contrasena);
    if (longitud >= MIN_LONGITUD && longitud <= MAX_LONGITUD) {
    printf("Longitud correcta\n");
    } else {
    printf("La longitud debe estar entre 8 y 20 caracteres\n");
    }
    if (strpbrk(contrasena, "ABCDEFGHIJKLMNOPQRSTUVWXYZ") != NULL)
        mayuscula = 1;
    if (mayuscula == 1) {
        printf("Contiene mayuscula\n");
    } else {
    printf("La contrasena debe contener al menos una mayuscula\n");
    }
    if (strpbrk(contrasena, "0123456789") != NULL)
        digito = 1;
    if (digito == 1) {
        printf("Contiene digito\n");
    } else {
        printf("La contrasena debe contener al menos un digito\n");
    }
    es_valida = (longitud >= MIN_LONGITUD && longitud <= MAX_LONGITUD && mayuscula == 1 && digito == 1);
    if (es_valida == 1)
        printf("Contrasena valida\n");
    if (es_valida == 0)
        printf("Contrasena no valida\n");
}
```

3.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'EJERCICIO191DIFICIL.exe'
Ingrese contrasena: Alison
La longitud debe estar entre 8 y 20 caracteres
Contiene mayuscula
La contrasena debe contener al menos un digito
Contrasena no valida
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'EJERCICIO191DIFICIL.exe'
Ingrese contrasena: Alison1234
Longitud correcta
Contiene mayuscula
Contiene digito
Contrasena valida
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'EJERCICIO191DIFICIL.exe'
Ingrese contrasena: Alisonnaguacruz
Longitud correcta
Contiene mayuscula
La contrasena debe contener al menos un digito
Contrasena no valida
```

4.1. Enunciado del algoritmo

Ejercicio192:

Construya un algoritmo que lea tres números reales q, c y dato2 por teclado. Use expresiones relacionales y lógicas para determinar cuál es el mayor, el menor y el del medio, sin usar estructuras condicionales (únicamente asignaciones con expresiones condicionales). Use constantes literales 0 para comparar y una constante simbólica UMBRAL = 10. Muestre por pantalla los resultados. Justifique el tipo de dato de cada variable auxiliar.

4.2. Análisis del problema

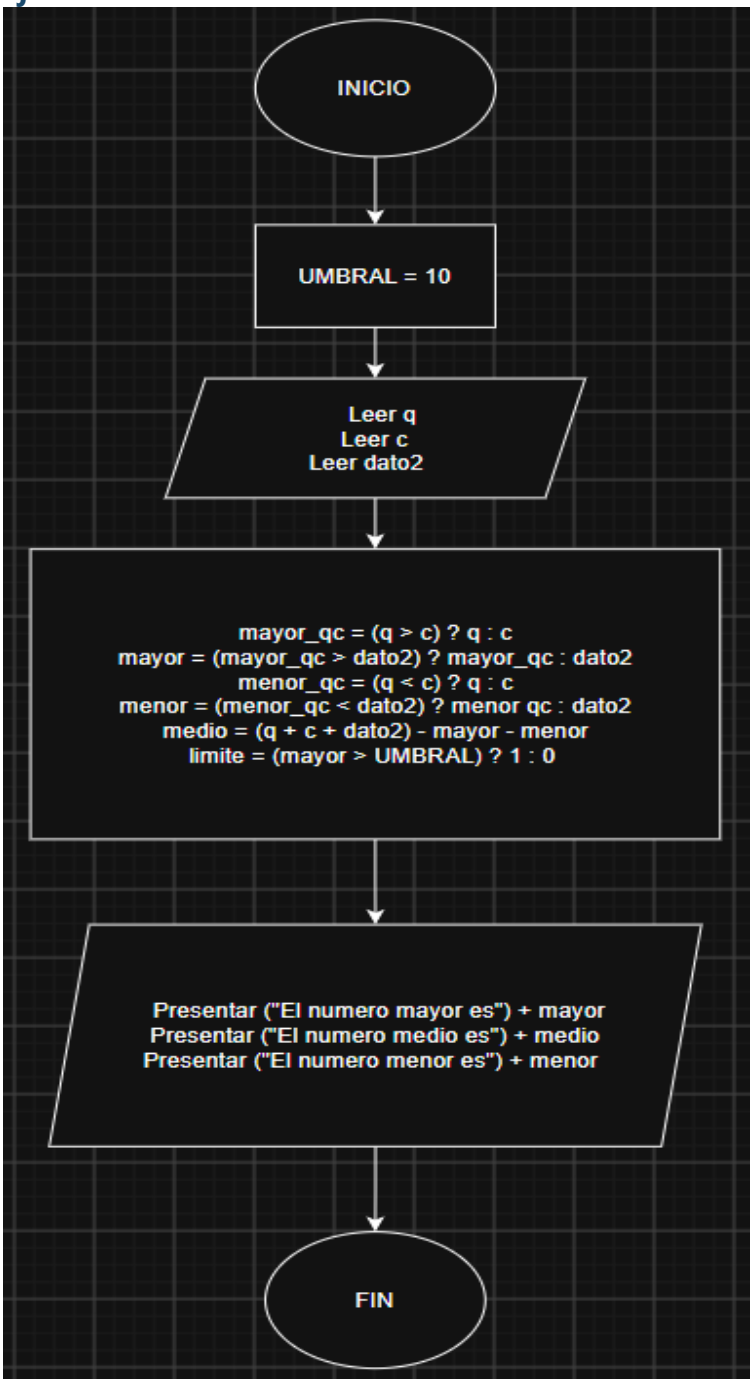
Datos de entrada

1. Valor de q
2. Valor de c
3. Valor de dato2

Datos de salida

1. El número mayor
2. El número menor
3. Número del medio
4. Mensaje indicando si el número supera al Umbral

4.3. Diagrama de flujo



4.4. Pseudocódigo

```
INICIO
//Definir variables
UMBRAL = 10 como constante;
Leer q como real;
Leer c como real;
Leer dato2 como real;
//Entrada
q = 0.0;
c = 0.0;
dato2 = 0.0;
//Proceso
mayor_qc = (q > c) ? q : c;
mayor = (mayor_qc > dato2) ? mayor_qc : dato2;
menor_qc = (q < c) ? q : c;
menor = (menor_qc < dato2) ? menor qc : dato2;
medio = (q + c + dato2) - mayor - menor;
limite = (mayor > UMBRAL) ? 1 : 0;
//Salida
Presentar ("El numero mayor es") + mayor;
Presentar ("El numero medio es") + medio;
Presentar ("El numero menor es") + menor;
FIN
```

4.5. Prueba de escritorio

Prueba	q	c	dato2	¿Mayor supera al UMBRAL?	Mayor	Menor	Medio
1	123	12	456	Sí	456.00	12.00	123.00
2	-12	-54	12	Sí	12.00	-54.00	-12.00
3	1	2	3	No	3.00	1.00	2.00

4.6. Codificación en C

EJERICICOSDIFICILES > C EJERCICIO192DIFICIL.c > ...

```c
#include <stdio.h>
int main () {

    int UMBRAL = 10;
    float q = 0.0, c = 0.0, dato2 = 0.0, mayor_qc = 0.0, mayor = 0.0, menor = 0.0, menor_qc = 0.0, medio = 0.0, limite = 0.0;
    printf("Ingrese el valor de q:");
    scanf("%f", &q);

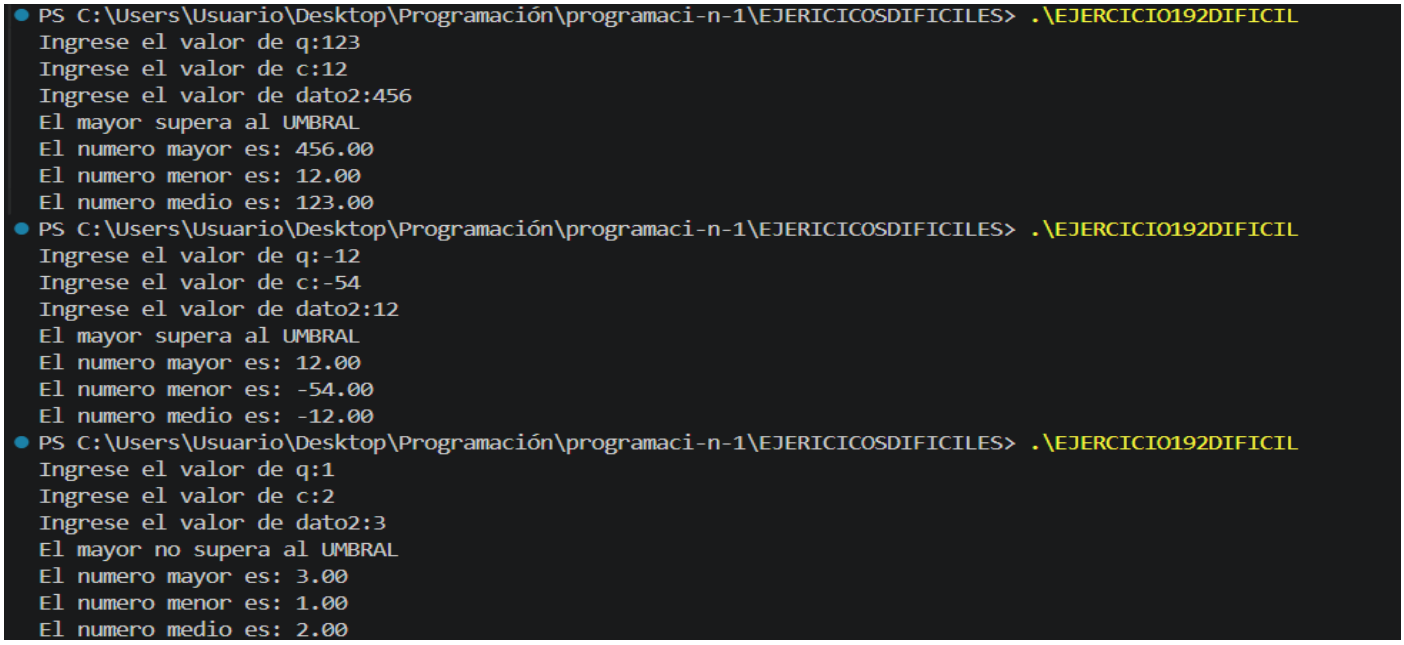
    printf("Ingrese el valor de c:");
    scanf("%f", &c);

    printf("Ingrese el valor de dato2:");
    scanf("%f", &dato2);
mayor_qc = (q > c)? q : c;
mayor = (mayor_qc > dato2)? mayor_qc : dato2;
menor_qc = (q < c)? q : c;
menor = (menor_qc < dato2)? menor_qc : dato2;
medio = (q + c + dato2) - mayor - menor;
if (limite = (mayor > UMBRAL)) {
printf("El mayor supera al UMBRAL\n");
} else {
    printf("El mayor no supera al UMBRAL\n");
}

 printf("El numero mayor es: %.2f\n", mayor);
 printf("El numero menor es: %.2f\n", menor);
 printf("El numero medio es: %.2f\n", medio);

}
```

4.7. Salida del programa



5.1. Enunciado del algoritmo

Ejercicio193

Construya un algoritmo que lea tres números reales q, c y dato2 por teclado. Use expresiones relacionales y lógicas para determinar cuál es el mayor, el menor y el del medio, sin usar estructuras condicionales (únicamente asignaciones con expresiones condicionales). Use constantes literales 0 para comparar y una constante simbólica UMBRAL = 10. Muestre por pantalla los resultados. Justifique el tipo de dato de cada variable auxiliar.

5.2. Análisis del problema

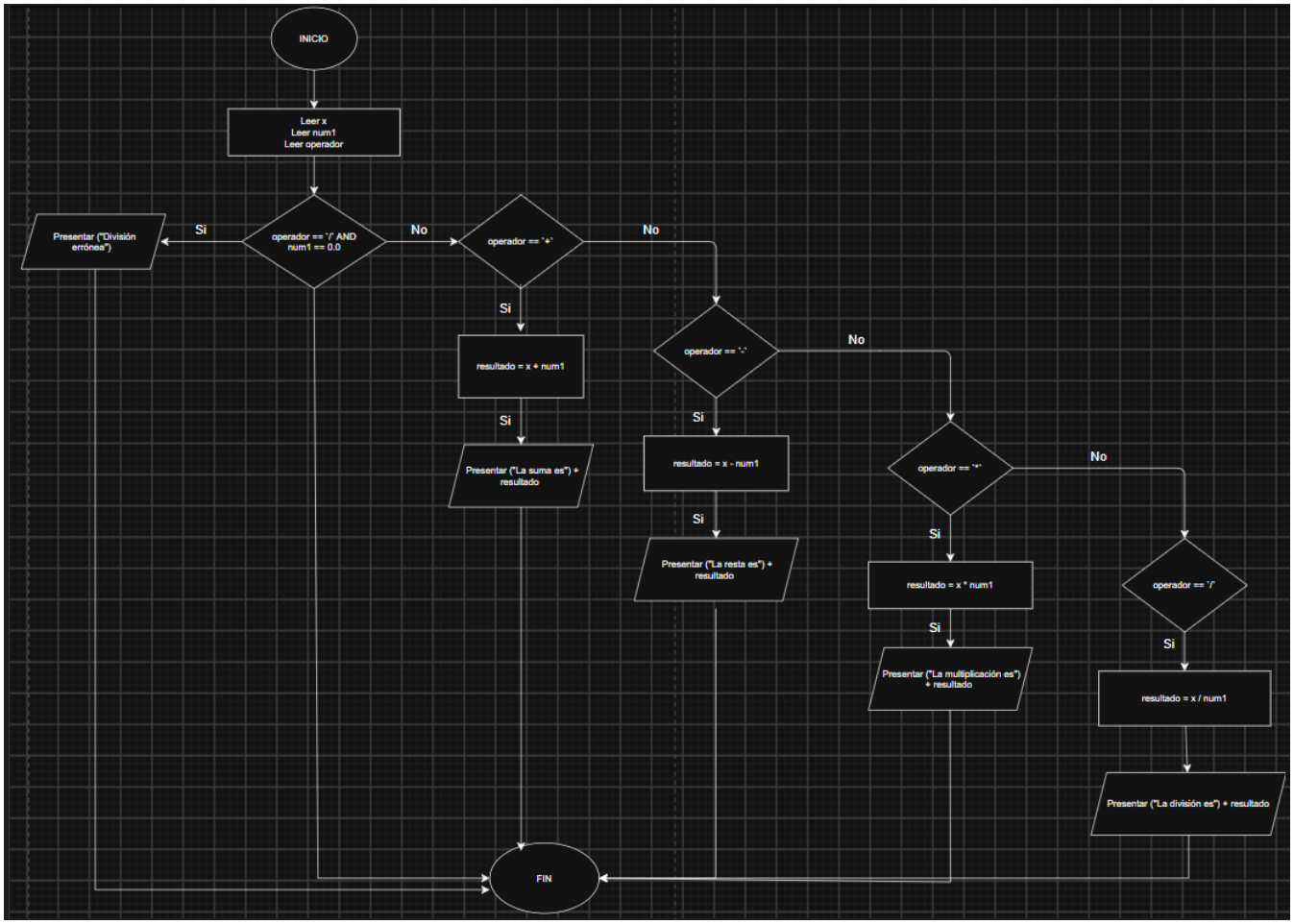
Datos de entrada

1. Valor de x
2. Valor de num1
3. Carácter del operador

Datos de salida

1. Si el operador es + sale el resultado de la suma
2. Si el operador es - sale el resultado de la resta
3. Si el operador es * sale el resultado de la multiplicación
4. Si el operador es / sale el resultado de la división
5. Si el operador es / y el num1 = 0.0 sale división errónea

5.3. Diagrama de flujo



5.4. Pseudocódigo

```
INICIO
//Definir variables
Leer x como real;
Leer num1 como real;
Leer operador como real;
//Entrada
x = 0.0, num1 = 0.0, resultado = 0.0;
char operador = ' ';
//Declaración del operador
Si (operador == `/` AND num1 == 0.0) Presentar ("División errónea");
Sino;
Si (operador == `+`) Entonces;
resultado = x + num1;
//salida
Presentar ("La suma es") + resultado;
FinSi;
Si (operador == `-`) Entonces;
resultado = x - num1;
//salida
Presentar ("La resta es") + resultado;
FinSi;
Si (operador == `*`) Entonces;
resultado = x * num1;
//salida
Presentar ("La multiplicación es") + resultado;
FinSi;
Si (operador == `/`) Entonces;
resultado = x / num1;
//salida
```

```
Presentar ("La división es") + resultado;
FIN
```

5.5. Prueba de escritorio

Prueba	x	num1	operador	Resultado
1	54	0	/	División errónea
2	25	78	+	La suma es 103.00
3	7	4	*	La multiplicación es 28.00

5.6. Codificación en C

```c
# include <stdio.h>
int main () {
float x = 0.0, num1 = 0.0, resultado = 0.0;
char operador = ' ';

printf("Ingrese el valor de x:");
scanf("%f", &x);

printf("Ingrese el valor de num1:");
scanf("%f", &num1);

printf("Ingrese el valor de operador:");
scanf(" %c", &operador);

if (operador == '/' && num1 == 0.0) {
printf("Division erronea\n");
} else {

    resultado = (operador == '+' ) ? (x + num1) :
    (operador == '-' ) ? (x - num1) :
    (operador == '*' ) ? (x * num1) :
    (operador == '/') ? (x / num1) : 0.0;
    printf("La %s es %2.2f\n",
    (operador == '+' ) ? "suma" :
    (operador == '-' ) ? "resta" :
    (operador == '*' ) ? "multiplicacion" :
    (operador == '/') ? "division" : "operacion invalida", resultado);
}
}
```

5.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\Unidad1> .\EJERCICIO193DIFICIL
Ingrese el valor de x:54
Ingrese el valor de num1:0
Ingrese el valor de operador:/
Division erronea
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\Unidad1> .\EJERCICIO193DIFICIL
Ingrese el valor de x:25
Ingrese el valor de num1:78
Ingrese el valor de operador:+
La suma es 103.00
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\Unidad1> .\EJERCICIO193DIFICIL
Ingrese el valor de x:7
Ingrese el valor de num1:4
Ingrese el valor de operador:*
La multiplicacion es 28.00
```

6.1. Enunciado del algoritmo

Ejercicio194

Cree un algoritmo que lea un número entero de tres cifras dato1 por teclado. Use expresiones aritméticas (operadores división entera y módulo con constantes literales 10 y 100) para extraer las unidades, decenas y centenas. Asigne cada una a variables enteras separadas (z, c, num2). Use una constante simbólica UMBRAL = 5 y evalúe la expresión lógica (unidades > UMBRAL OR decenas > UMBRAL OR centenas > UMBRAL). Muestre por pantalla el número original, sus cifras y el resultado de la expresión.

6.2. Análisis del problema

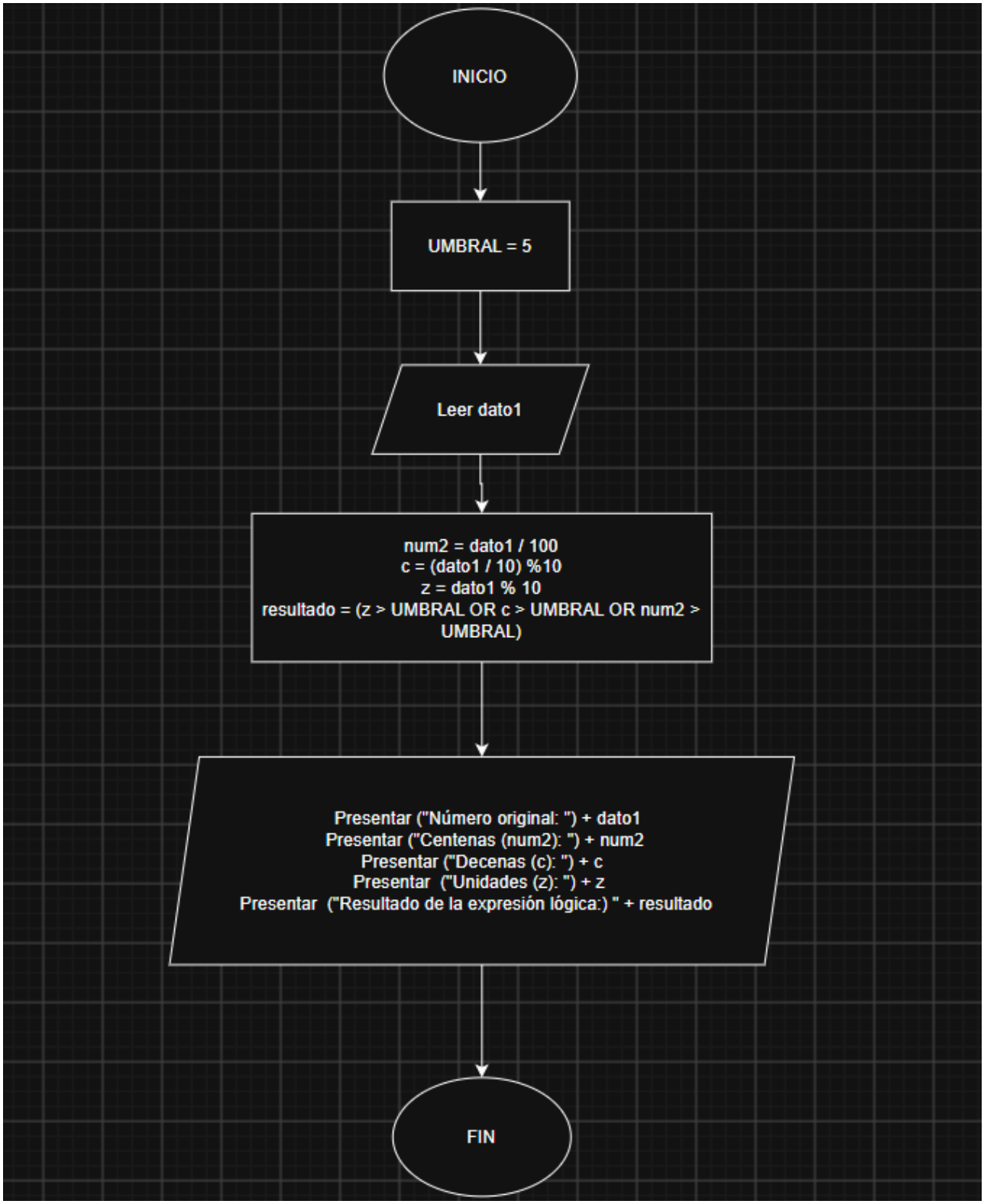
Datos de entrada

1. Valor del dato1

Datos de salida

1. El número mayor
2. El número menor
3. Número del medio
4. Mensaje indicando si el número supera al Umbral

6.3. Diagrama de flujo



6.4. Pseudocódigo

```
INICIO
//Definir variables
UMBRAL = 5 como constante;
Leer dato1 como entero;
//Entrada
dato1 = 0;
//Proceso
num2 = dato1 / 100;
c = (dato1 / 10) % 10;
```

```
z = dato1 % 10;
resultado = (z > UMBRAL OR c > UMBRAL OR num2 > UMBRAL);
//Salida
Presentar ("Número original:") + dato1;
Presentar ("Centenas:") + num2;
Presentar ("Decenas:") + c;
Presentar ("Unidades:") + z;
Presentar ("Resultado de la expresión lógica:) " + resultado;
FIN
```

6.5. Prueba de escritorio

Prueba	Número ingresado	Número original	Centenas	Decenas	Unidades	Resultado de la expresión lógica
1	134	134	1	3	4	0
2	489	489	4	8	9	1

6.6. Codificación en C

```c
#include <stdio.h>

int main () {;
    int UMBRAL = 5, dato1 = 0, num2 = 0, c = 0, z = 0, resultado = 0;

    printf("Porfavor ingrese valor de tres cifras:");
    scanf("%d", &dato1);

    num2 = dato1 / 100;
    c = (dato1 / 10) % 10;
    z = dato1 % 10;
    resultado= (z > UMBRAL || c > UMBRAL || num2 > UMBRAL);
printf("Numero original: %d \n", dato1);
printf("Centenas: %d \n", num2);
printf("Decenas: %d \n", c);
printf("Unidades: %d \n", z);
printf("Resultado de la expresion logica: %d \n", resultado);

}
```

6.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERICICOSDIFICILES\output> & .\'EJERCICIO194DIFICIL.exe'
Porfavor ingrese valor de tres cifras:134
Numero original: 134
Centenas: 1
Decenas: 3
Unidades: 4
Resultado de la expresion logica: 0
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERICICOSDIFICILES\output> & .\'EJERCICIO194DIFICIL.exe'
Porfavor ingrese valor de tres cifras:489
Numero original: 489
Centenas: 4
Decenas: 8
Unidades: 9
Resultado de la expresion logica: 1
```

7.1. Enunciado del algoritmo

Ejercicio195

Diseñe un algoritmo que lea la fecha de nacimiento de una persona: dia (entero), mes (entero) y año (entero). Use constantes simbólicas para los límites válidos: DIA_MAX = 31, MES_MAX = 12, AÑO_MIN = 1900, AÑO_MAX = 2025. Evalúe la expresión lógica compuesta: (dia >= 1

AND dia <= DIA_MAX AND mes >= 1 AND mes <= MES_MAX AND año >= AÑO_MIN AND año <= AÑO_MAX). Asigne el resultado a fecha_valida (booleano). Calcule la edad aproximada con la expresión aritmética (2025 - año). Muestre por pantalla la validación y la edad.

7.2. Análisis del problema

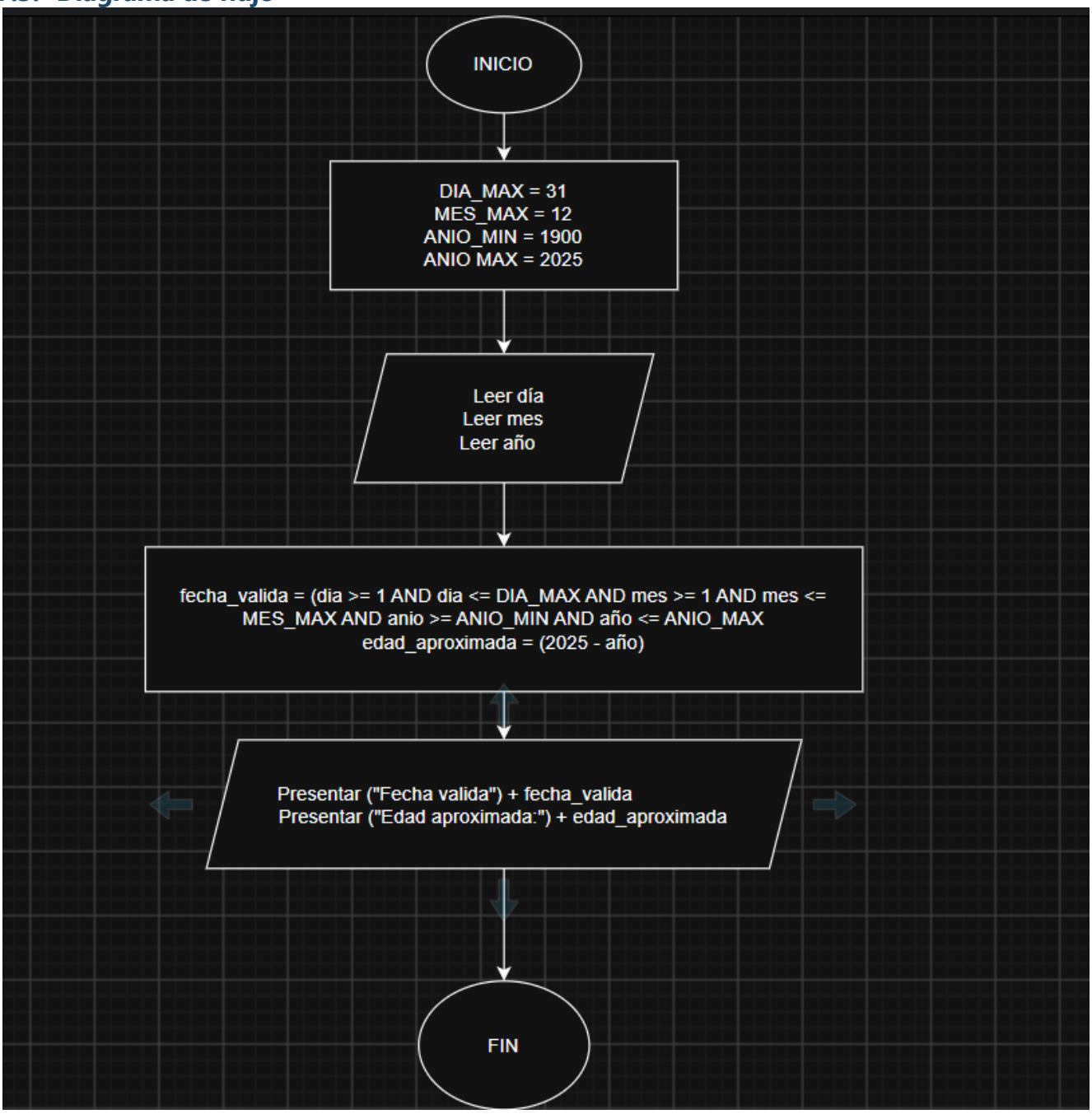
Datos de entrada

1. Valor del día
2. Valor del mes
3. Valor de

Datos de salida

1. La fecha valida
2. La edad aproximada

7.3. Diagrama de flujo



7.4. Pseudocódigo

```
INICIO
//Definir variables
DIA_MAX = 31 como constante;
MES_MAX = 12 como constante;
ANIO_MIN = 1900 como constante;
ANIO_MAX = 2025 como constante;
Leer día como entero;
Leer mes como entero;
Leer año como entero;
//Entrada
dia = 0;
mes = 0;
año = 0;
//Proceso
fecha_valida = (dia >= 1 AND dia <= DIA_MAX AND mes >= 1 AND mes <= MES_MAX AND anio >= ANIO_MIN
AND año <= ANIO_MAX;
edad_aproximada = (2025 - año);
//Salida
Presentar ("Fecha valida") + fecha_valida;
Presentar ("Edad aproximada:") + edad_aproximada;
FIN
```

7.5. Prueba de escritorio

Prueba	Día	Mes	Año	¿Fecha válida?	Edad aproximada
1	15	12	2008	Sí	17
2	14	1	1899	No	126

7.6. Codificación en C

```c
# include <stdio.h>
int main () {
int DIA_MAX = 31;
int MES_MAX = 12;
int ANIO_MIN = 1900;
int ANIO_MAX = 2025;
int dia = 0;
int mes = 0;
int anio = 0;
int fecha_valida = 0;
int edad_aproximada = 0;

printf("Porfavor ingrese el dia:");
scanf("%d", &dia);

printf("Porfavor ingrese el mes:");
scanf("%d", &mes);

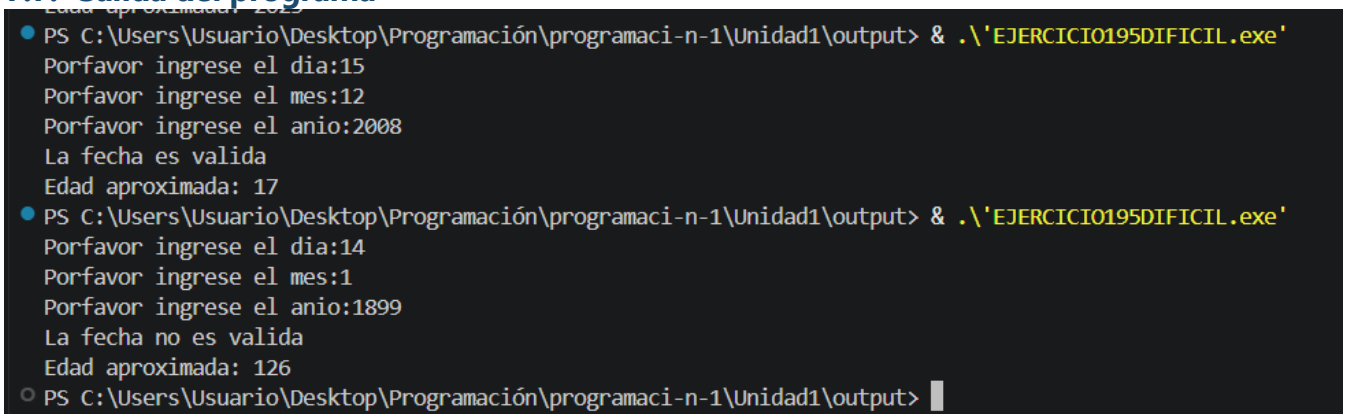
printf("Porfavor ingrese el anio:");
scanf("%d", &anio);

fecha_valida = (dia >= 1 && dia<= DIA_MAX && mes >= 1 && mes <= MES_MAX && anio >= ANIO_MIN && anio <= ANIO_MAX);
edad_aproximada = 2025 - anio;

if (fecha_valida == 1) {
    printf("La fecha es valida\n");
} else {
    printf("La fecha no es valida\n");
}
printf("Edad aproximada: %d\n", edad_aproximada);

}
```

7.7. Salida del programa



EJERCICIOS NIVEL MEDIO

8.1. Enunciado del algoritmo

Ejercicio16

En un taller mecanico, cree un algoritmo que lea tres notas (reales) y sus respectivos porcentajes (reales) que suman 100. Use la constante literal 100 como divisor de verificacion. Calcule el promedio ponderado con la expresion aritmetica (nota1 * p1 + nota2 * p2 + nota3 * p3) / 100. Asignelo a promedio_ponderado. Muestre por pantalla el resultado. Justifique el uso de constantes simbolicas para los porcentajes. Ademas del calculo principal, determine si el resultado es positivo o negativo. Agregue un mensaje indicando el tipo de operacion realizada.

8.2. Análisis del problema

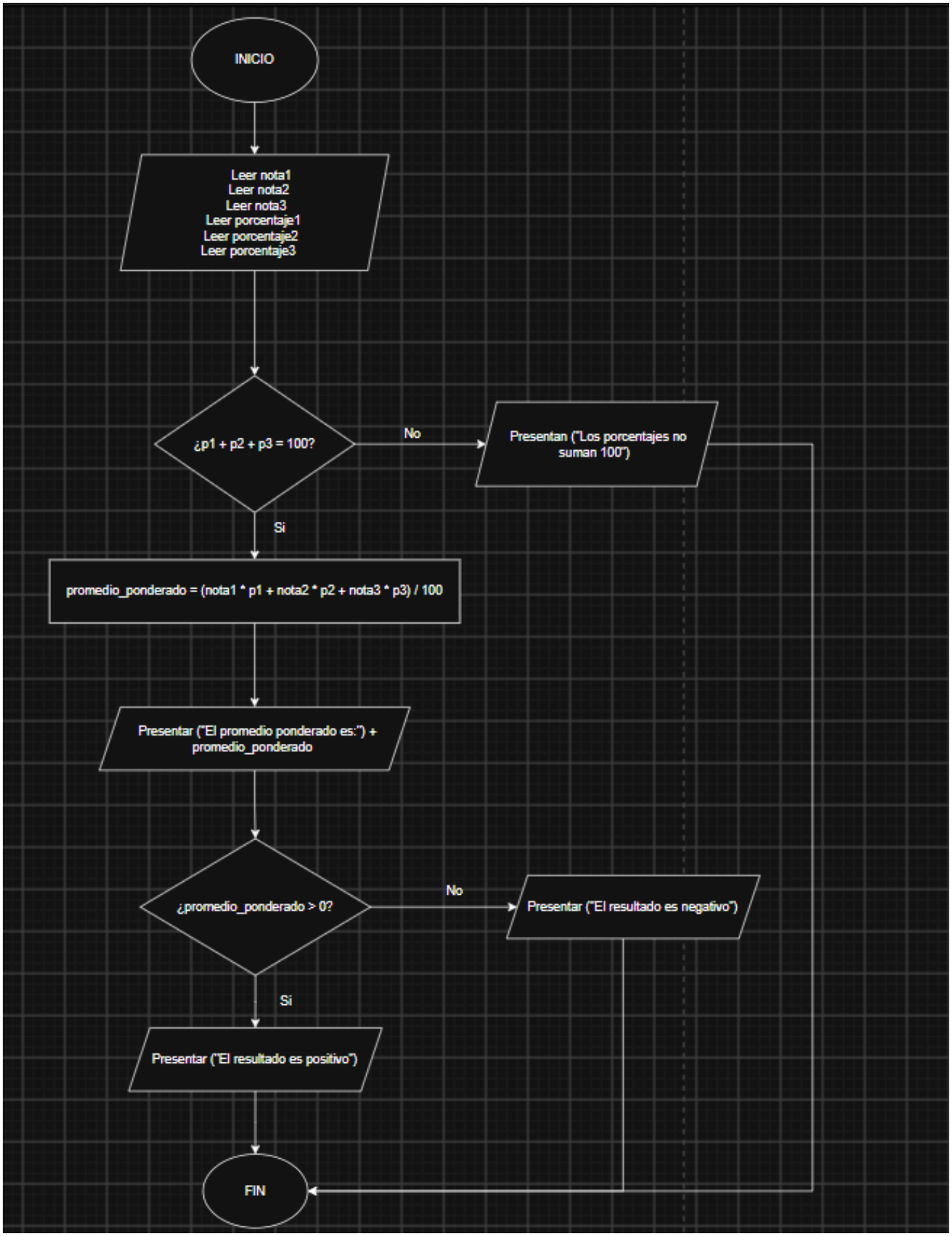
Datos de entrada

1. Valor de la nota1
2. Valor de la nota2
3. Valor de la nota3
4. Valor del porcentaje1
5. Valor del porcentaje1
6. Valor del porcentaje3

Datos de salida

1. El promedio ponerado
2. Si el valor es negativo o positivo

8.3. Diagrama de flujo



8.4. Pseudocódigo

```
INICIO
```

```
//Definir variables
Leer nota1 como real;
Leer nota2 como real;
Leer nota3 como real;
Leer porcentaje1 como real;
Leer porcentaje2 como real;
Leer porcentaje3 como real;
//Entrada
nota1 = 0.0;
nota2 = 0.0;
nota3 = 0.0;
p1 = 0.0;
p2 = 0.0;
p3 = 0.0;
//Proceso
SI p1 + p2 + p3 = 100
promedio_ponderado = (nota1 * p1 + nota2 * p2 + nota3 * p3) / 100;
//Salida
Presentar ("El promedio ponderado es:") + promedio_ponderado;
Sino
Presentan ("Los porcentajes no suman 100");
FIN_SI
SI promedio_ponderado > 0
//Salida
Presentar ("El resultado es positivo");
Sino
Presentar ("El resultado es negativo");
FIN
```

8.5. Prueba de escritorio

Prueba	nota1	nota2	nota3	porcentaje1	porcentaje2	porcentaje3	Suma de porcentajes	Promedio ponderado	Mensaje de validación	Resultado
1	90	80	85	50	30	50	130	No se calcula	Error los porcentajes no suman 100	El resultado es negativo
2	50	60	65	60	50	1	111	No se calcula	Error los porcentajes no suman 100	El resultado es negativo
3	50	60	55	50	25	25	100	53.75	Ninguno	El resultado es positivo

8.6. Codificación en C

```c
#include <stdio.h>
int main () {
    float nota1 = 0.0, nota2 = 0.0, nota3 = 0.0, p1 = 0.0, p2 = 0.0, p3 = 0.0, promedio_ponerado = 0.0;
    printf ("Ingrese la nota1:");
    scanf("%f", &nota1);
    printf ("Ingrese la nota2:");
    scanf("%f", &nota2);
    printf ("Ingrese la nota3:");
    scanf("%f", &nota3);
    printf ("Ingrese el porcentaje1:");
    scanf("%f", &p1);
    printf ("Ingrese el porcentaje2:");
    scanf("%f", &p2);
    printf ("Ingrese el porcentaje3:");
    scanf("%f", &p3);

    if (p1 + p2 + p3 == 100) {
        promedio_ponerado = (nota1 * p1 + nota2 * p2 + nota3 * p3) / 100;
        printf("El promedio ponerado es:%.2f\n", promedio_ponerado);
    } else {
        printf("Error los porcentajes no suman 100\n");
    }

    if (promedio_ponerado > 0) {
        printf("El resultado es positivo\n");
    } else {
        printf("El resultado es negativo\n");
    }
    }
```

8.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio16.exe'
Ingrese la nota1:90
Ingrese la nota2:80
Ingrese la nota3:85
Ingrese el porcentaje1:50
Ingrese el porcentaje2:30
Ingrese el porcentaje3:50
Error los porcentajes no suman 100
El resultado es negativo
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio16.exe'
Ingrese la nota1:50
Ingrese la nota2:60
Ingrese la nota3:65
Ingrese el porcentaje1:60
Ingrese el porcentaje2:50
Ingrese el porcentaje3:1
Error los porcentajes no suman 100
El resultado es negativo
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio16.exe'
Ingrese la nota1:50
Ingrese la nota2:60
Ingrese la nota3:55
Ingrese el porcentaje1:50
Ingrese el porcentaje2:25
Ingrese el porcentaje3:25
El promedio ponerado es:53.75
El resultado es positivo
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output>
```

9.1. Enunciado del algoritmo

Ejercicio39

En una escuela, diseñe un algoritmo que lea la distancia recorrida (real, en metros) y el tiempo (real, en segundos). Calcule la velocidad con la expresion aritmetica (distancia / tiempo) y asignela a velocidad. Evalúe la expresion logica (tiempo > 0 AND distancia >= 0). Muestre por pantalla la velocidad en m/s. Justifique el tipo de dato de cada variable. Ademas del calculo principal, multiplique el resultado por 1.5 como bono. Agregue validaciones para que no haya resultados negativos.

9.2. Análisis del problema

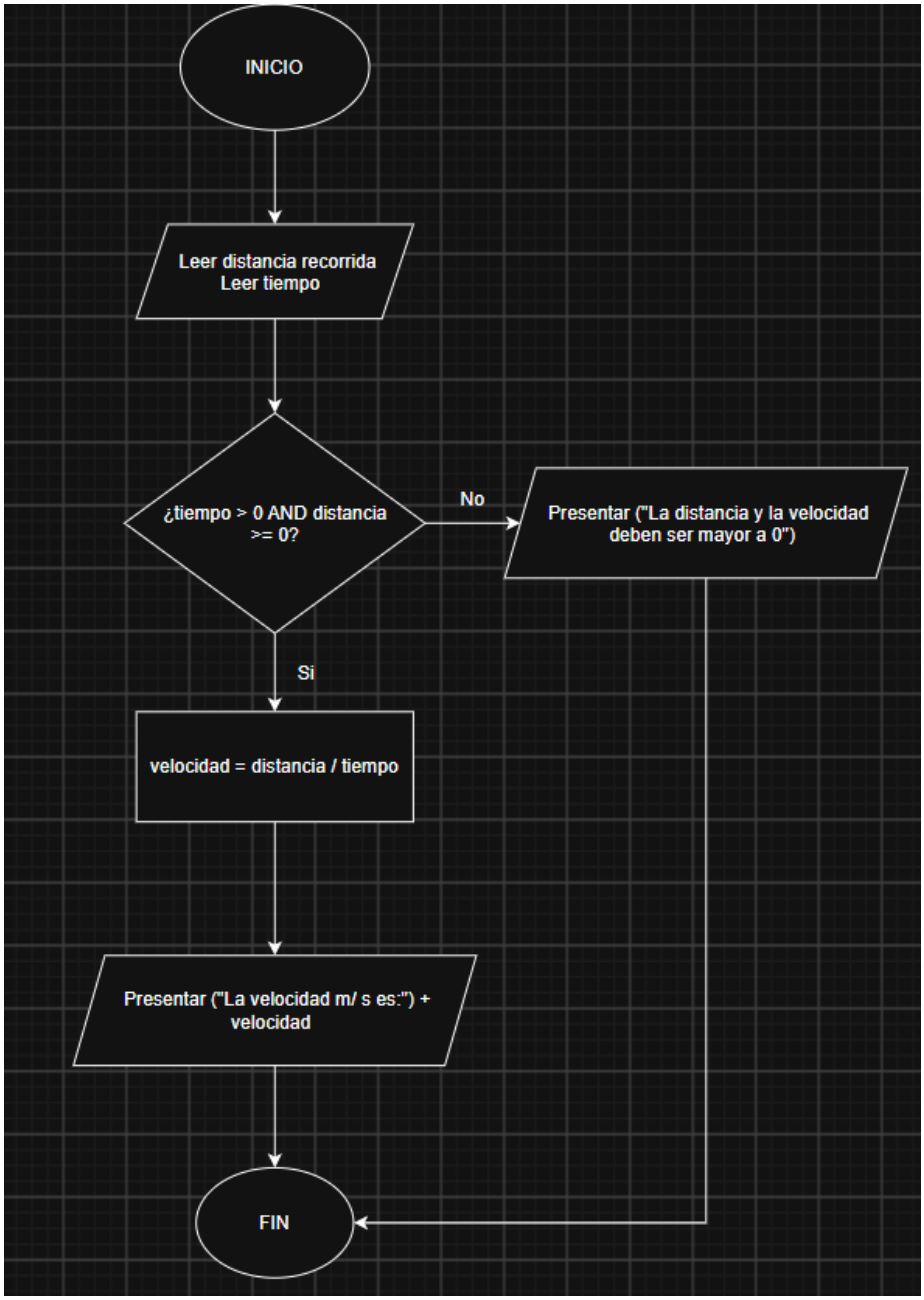
Datos de entrada

1. Valor de distancia
2. Valor del tiempo

Datos de salida

1. La velocidad en m/s

9.3. Diagrama de flujo



9.4. Pseudocódigo

```
INICIO
//Definir variables
Leer distancia recorrida como real;
Leer tiempo como real;
//Entrada
distancia = 0.0
tiempo = 0.0
//Proceso
SI tiempo > 0 AND distancia >= 0
velocidad = distancia / tiempo;
//Salida
Presentar ("La velocidad m/s es:") + velocidad;
Sino
Presentar ("La distancia y la velocidad deben ser mayor a 0");
FIN_SI
FIN
```

9.5. Prueba de escritorio

Prueba	distancia	tiempo	¿tiempo > 0 y distancia >= 0?	Velocidad (m/s)	Salida esperada
1	100	10	Sí	10.00	La velocidad en m/s es: 10.00m/s
2	50	4	Sí	12.50	La velocidad en m/s es: 12.50m/s
3	10	3	Sí	3.33	La velocidad en m/s es: 3.33m/s
4	0	5	Sí	0.00	La velocidad en m/s es: 0.00m/s

9.6. Codificación en C

```c
#include <stdio.h>
int main () {
    float distancia = 0.0, tiempo = 0.0, velocidad = 0.0;

printf("Ingrese la distancia recorrida: ");
scanf("%f", &distancia);
printf("Ingrese el tiempo: ");
scanf("%f", &tiempo);

if (tiempo > 0 && distancia >= 0) {
    velocidad = distancia / tiempo;
    printf("La velocidad en m/s es: %.2fm/s\n", velocidad);

} else {
    printf("La distancia y la velocidad deben ser mayor a 0\n");
}
}
```

9.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio39.exe'
Ingrese la distancia recorrida: 100
Ingrese el tiempo: 10
La velocidad en m/s es: 10.00m/s
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio39.exe'
Ingrese la distancia recorrida: 50
Ingrese el tiempo: 4
La velocidad en m/s es: 12.50m/s
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio39.exe'
Ingrese la distancia recorrida: 10
Ingrese el tiempo: 3
La velocidad en m/s es: 3.33m/s
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio39.exe'
Ingrese la distancia recorrida: 0
Ingrese el tiempo: 5
La velocidad en m/s es: 0.00m/s
```

10.1. Enunciado del algoritmo

Ejercicio 40

En una joyeria, cree un algoritmo que lea las dos bases (reales) y la altura (real) de un trapecio. Use la constante literal 2 en el denominador. Calcule el area con la expresion aritmetica ((base_mayor + base_menor) * altura / 2) y asignela a area. Evalúe la expresion logica (base_mayor > base_menor AND altura > 0). Muestre por pantalla el area calculada. Ademas

del calculo principal, calcule el resultado elevado al cubo. Incluya mensajes informativos durante la ejecucion del algoritmo.

10.2. Análisis del problema

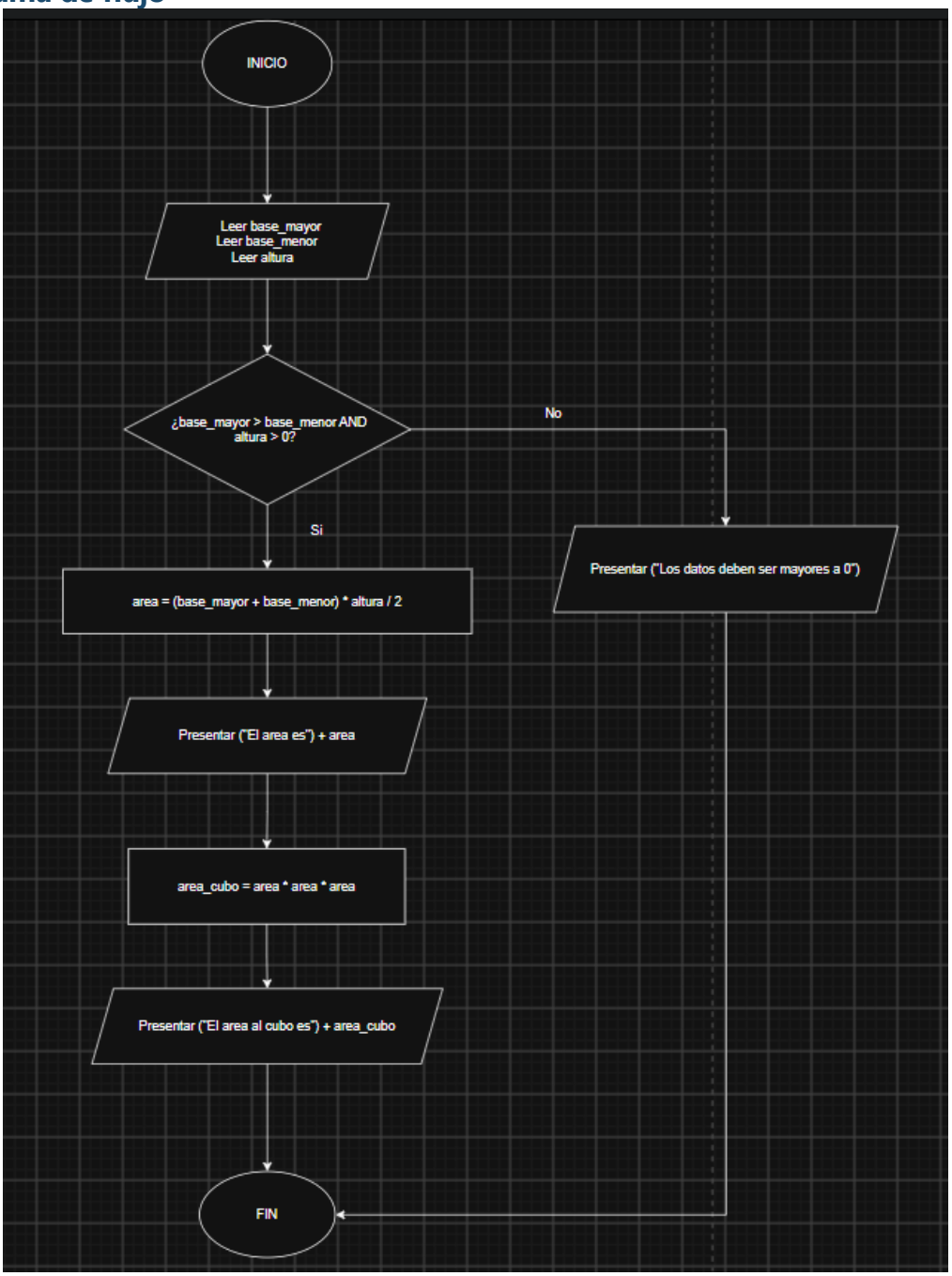
Datos de entrada

1. Valor de base mayor
2. Valor de la base menor
3. Valor de la altura

Datos de salida

1. Valor del área
2. Valor del área al cubo

10.3. Diagrama de flujo



10.4. Pseudocódigo

```
INICIO
//Definir variables
Leer base_mayor como real;
Leer base_menor como real;
Leer altura como real;
//Entrada
base_mayor = 0.0;
base_menor = 0.0;
altura = 0.0;
//Proceso
area = (base_mayor + base_menor) * altura / 2;
SI base_mayor > base_menor AND altura > 0
//Salida
Presentar ("El area es") + área;
Sino
Presentar ("Los datos deben ser mayores a 0");
FIN_SI
area_al_cubo = area * area * área;
//Salida
Presentar ("El area al cubo es") + area_cubo;
FIN
```

10.5. Prueba de escritorio

Prueba	Base mayor	Base menor	Altura	Área	Área al cubo
1	70	50	35	2100.00	9260999680.00
2	43	25	7	238.00	13481272.00
3	67	43	14	770.00	456532992.00

10.6. Codificación en C

```c
#include <stdio.h>
int main () {
float base_mayor = 0.0, base_menor = 0.0, altura = 0.0;
float area = 0.0, area_cubo = 0.0;

printf("Ingrese el valor de la base mayor:");
scanf("%f", &base_mayor);

printf("Ingrese el valor de la base menor:");
scanf("%f", &base_menor);

printf("Ingrese la altura:");
scanf("%f", &altura);

area = (base_mayor + base_menor) * altura / 2;

if (base_mayor > base_menor && altura > 0) {
printf("El area es:%.2f\n", area);
area_cubo = area * area * area;
printf("El area al cubo es:%.2f\n", area_cubo);
} else {
printf("La base mayor debe ser mayor a la base menor y la altura debe ser mayor a 0\n");
}
}
```

10.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS> .\Ejercicio40
Ingrese el valor de la base mayor:70
Ingrese el valor de la base menor:50
Ingrese la altura:35
El area es:2100.00
El area al cubo es:9260999680.00
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS> .\Ejercicio40
Ingrese el valor de la base mayor:43
Ingrese el valor de la base menor:25
Ingrese la altura:7
El area es:238.00
El area al cubo es:13481272.00
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS> .\Ejercicio40
Ingrese el valor de la base mayor:67
Ingrese el valor de la base menor:43
Ingrese la altura:14
El area es:770.00
El area al cubo es:456532992.00
```

11.1. Enunciado del algoritmo

Ejercicio 41

En un cine, construya un algoritmo que lea un volumen en galones (real). Use una constante simbolica GALON_LITRO = 3.78541. Calcule la equivalencia en litros con la expresion aritmetica (galones * GALON_LITRO) y asignela a litros. Evalúe la expresion relacional (litros > 0). Muestre por pantalla 'X galones equivalen a Y litros'. Ademas del calculo principal, calcule el producto del resultado por PI. Agregue un mensaje indicando el tipo de operacion realizada.

11.2. Análisis del problema

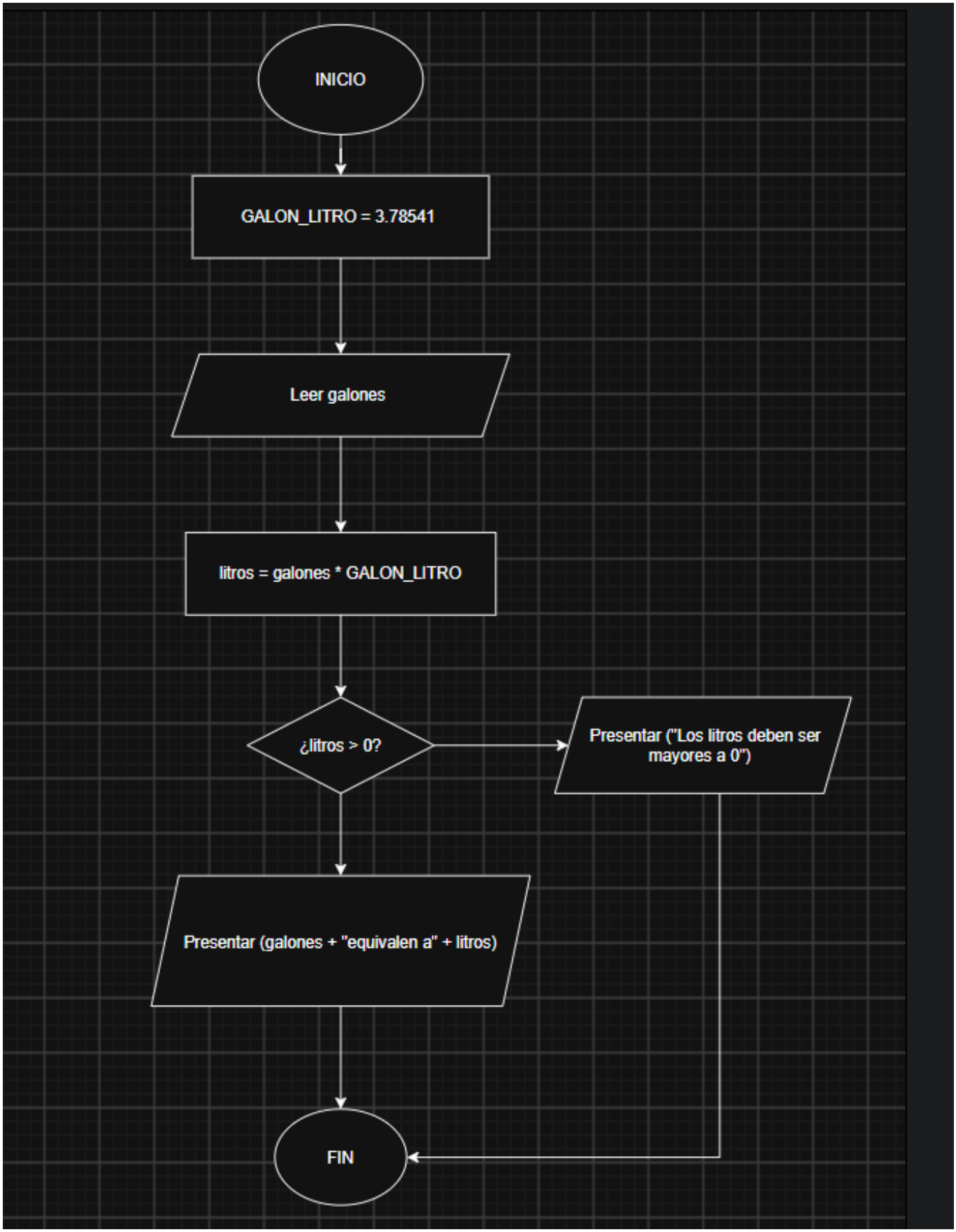
Datos de entrada

1. Valor del volumen en galones

Datos de salida

1. Equivalencia de los galones en litros
2. El resultado multiplicado por PI

11.3. Diagrama de flujo



11.4. Pseudocódigo

```
INICIO
//Definir variables
```

```
GALON_LITRO = 3.78541 como constante;
Leer galones como real;
//Entrada
galones = 0.0;
//Proceso
litros = galones * GALON_LITRO;
SI litros > 0
//Salida
Presentar (galones + "equivalen a" + litros);
Sino
Presentar ("Los litros deben ser mayores a 0");
FIN
```

11.5. Prueba de escritorio

Prueba	Galones	Mensaje de conversión	Litros por PI
1	125	125.00 galones equivalen a 473.18 litros	1486.53
2	57	57.00 galones equivalen a 215.77 litros	677.86
3	0	Los litros deben ser mayores a 0	0.00

11.6. Codificación en C

```c
#include <stdio.h>
#include <math.h>
int main () {
float GALON_LITRO = 3.78541;
float litros = 0.0, galones = 0.0, resultado = 0.0;

printf("Ingrese el valor de el volumen en galones:");
scanf("%f", &galones);

litros = (galones * GALON_LITRO);

if (litros > 0) {
printf("%.2f galones equivalen a %.2f litros\n", galones, litros);
} else {
printf("Los litros deben ser mayores a 0\n");
}

resultado = litros * M_PI;
printf("El resultado de los litros por PI es:%.2f\n", resultado);
}
```

11.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio41.exe'
Ingrese el valor de el volumen en galones:125
125.00 galones equivalen a 473.18 litros
El resultado de los litros por PI es:1486.53
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio41.exe'
Ingrese el valor de el volumen en galones:57
57.00 galones equivalen a 215.77 litros
El resultado de los litros por PI es:677.86
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio41.exe'
Ingrese el valor de el volumen en galones:0
Los litros deben ser mayores a 0
El resultado de los litros por PI es:0.00
```

12.1. Enunciado del algoritmo

Ejercicio 50

En una cooperativa, construya un algoritmo que lea una altura en pies (real). Use una constante simbolica PIE_METRO = 0.3048. Calcule la equivalencia en metros con la expresion aritmetica (pies * PIE_METRO) y asignela a metros. Evalúe la expresion relacional (metros > 0). Muestre por pantalla 'X pies equivalen a Y metros'. Ademas, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecucion del algoritmo.

12.2. Análisis del problema

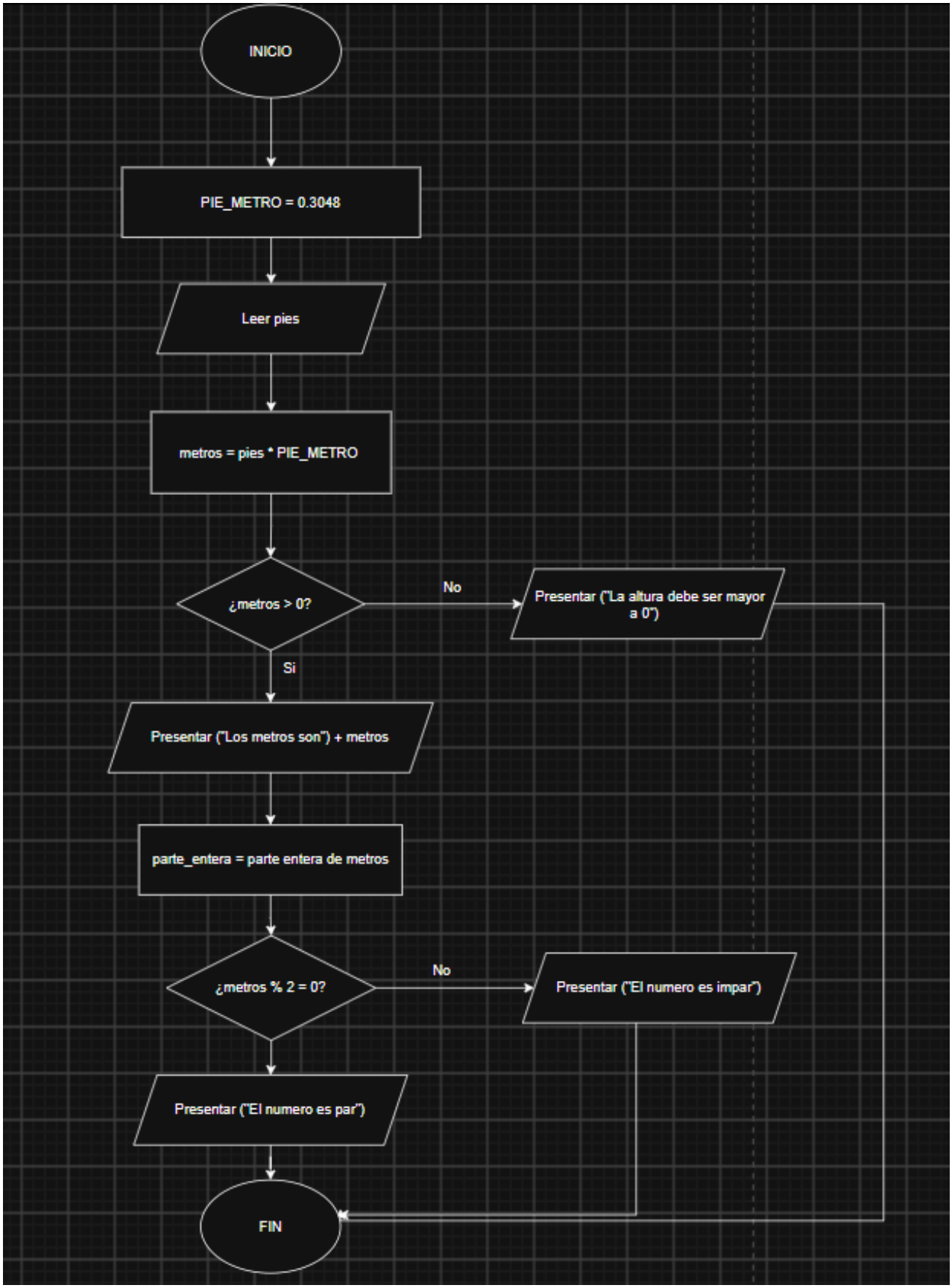
Datos de entrada

1. Valor de los pies

Datos de salida

1. Valor de los pies en metros
2. Si el número es par o impar

12.3. Diagrama de flujo



12.4. Pseudocódigo

```
INICIO
//Definir variables
PIE_METRO = 0.3048 como constante;
Leer pies como real;
```

```
//Entrada
pies = 0.0;
//Proceso
metros = pies * PIE_METRO;
SI metros > 0
//Salida
Presentar ("Los metros son") + metros;
Sino
Presentar ("La altura debe ser mayor a 0");
FIN_SI
parte_entera = parte entera de metros;
SI parte_entera % 2 == 0
//Salida
Presentar ("El numero es par");
Sino
Presentar ("El numero es impar");
FIN_SI
FIN
```

12.5. Prueba de escritorio

Prueba	Altura en pies	Mensaje de conversión	Parte entera	¿Par o impar?
1	123	123.00 pies equivalen a 37.49 metros	37	Impar
2	23	23.00 pies equivalen a 7.01 metros	7	Impar
3	0	La altura debe ser mayor a 0	No se muestra	No se muestra

12.6. Codificación en C

```c
#include <stdio.h>
int main () {
int parte_entera = 0;
float PIE_METRO = 0.3048, pies = 0.0, metros = 0;
printf("Ingrese la altura en pies:");
scanf("%f", &pies);

metros = pies * PIE_METRO;
// metros > 0
if (metros > 0) {
printf("%.2f pies equivalen a %.2f metros\n", pies, metros);

// cambiar a entero
parte_entera = (int) metros;
printf("La parte entera del resultado (%d)\n", parte_entera);

if (parte_entera % 2 == 0) {
     printf("El numero es par\n");
} else {
     printf("El numero es impar\n");

    }
} else {
     printf("La altura debe ser mayor a 0\n");
}
}
```

12.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio50.exe'
Ingrese la altura en pies:123
123.00 pies equivalen a 37.49 metros
La parte entera del resultado (37)
El numero es impar
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio50.exe'
Ingrese la altura en pies:23
23.00 pies equivalen a 7.01 metros
La parte entera del resultado (7)
El numero es impar
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio50.exe'
Ingrese la altura en pies:0
La altura debe ser mayor a 0
```

13.1. Enunciado del algoritmo

Ejercicio 101

En una universidad, diseñe un algoritmo que lea dos numeros reales por teclado. Use la constante literal 2 como divisor. Calcule el promedio con la expresion aritmetica ((num1 + num2) / 2) y asignelo a promedio. Evalúe la expresion relacional (promedio > 0). Muestre por pantalla el promedio calculado. Identifique el tipo de dato de cada variable. Adicionalmente, calcule la potencia del resultado elevado a la 4. Incluya una seccion que explique brevemente el proposito del algoritmo.

13.2. Análisis del problema

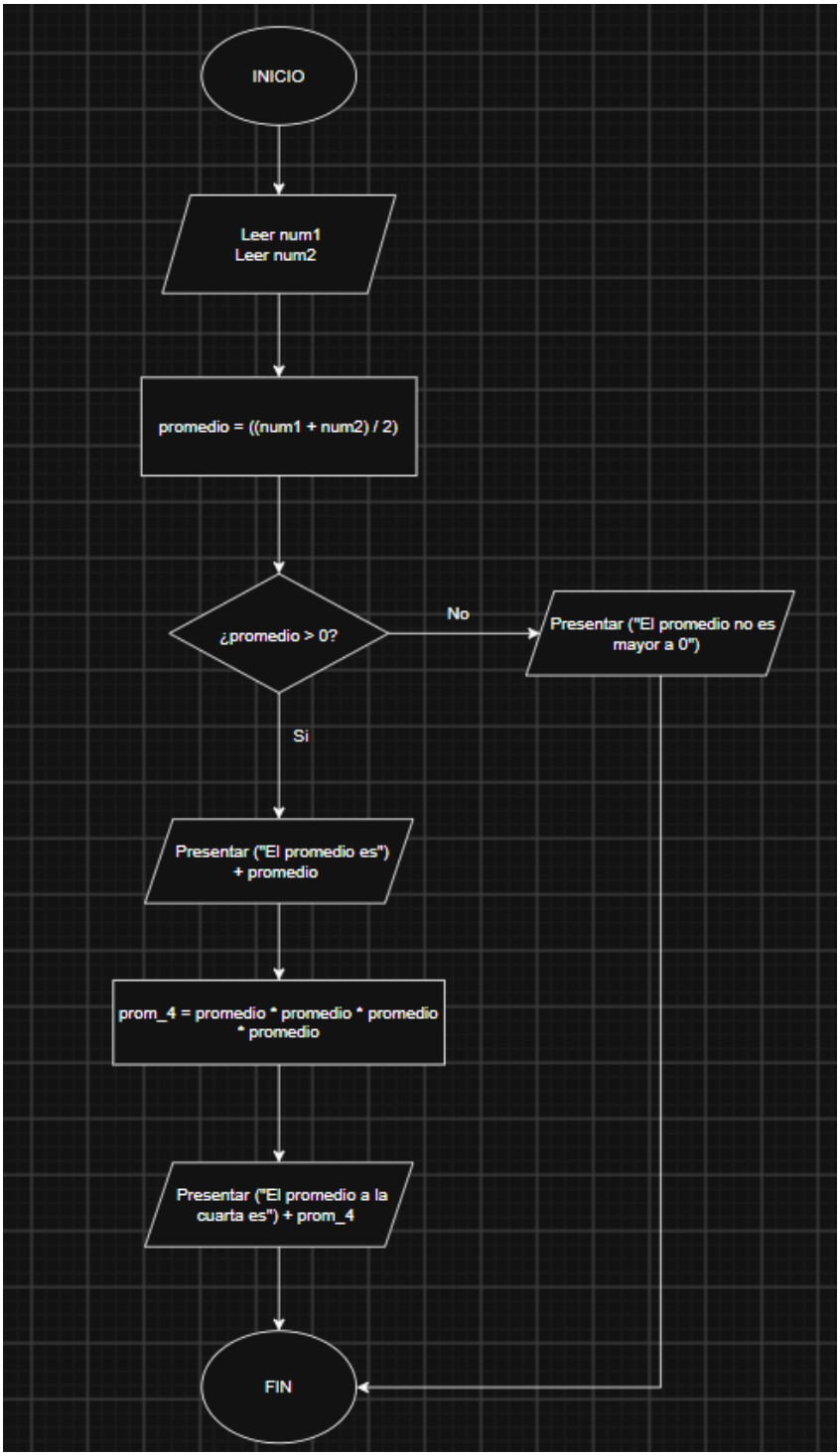
Datos de entrada

1. Valor de num1
2. Valor de num2

Datos de salida

1. Promedio calculado
2. Promedio calculado elevado a la 4

13.3. Diagrama de flujo



13.4. Pseudocódigo

```
INICIO
//Declarar variables
Leer num1 como real;
Leer num2 como real;
//Entrada
num1 = 0.0;
num2 = 0.0;
//Proceso
promedio = ((num1 + num2) / 2);
SI ¿promedio > 0?
//Salida
Presentar ("El promedio es") + promedio;
```

```
Sino
Presentar ("El promedio no es mayor a 0");
FIN_SI
prom_4 = promedio * promedio * promedio * promedio;
//Salida
Presentar ("El promedio a la cuarta es") + prom_4;
FIn
```

13.5. Prueba de escritorio

Prueba	num1	num2	Promedio	Promedio a la cuarta	Mensaje
1	30	71	50.50	6503775.00	Ninguno
2	4	0	2.00	16.00	Ninguno
3	0	0	No se muestra	No se muestra	El promedio no es mayor a 0

13.6. Codificación en C

```c
#include <stdio.h>
int main () {
float num1 = 0.0, num2 = 0.0, promedio = 0.0, prom_4 = 0.0;

printf("Ingrese el valor de num1:");
scanf("%f", &num1);
printf("Ingrese el valor de num2:");
scanf("%f", &num2);

promedio = ((num1 + num2) / 2);

if (promedio > 0) {
printf("El promedio es:%.2f\n", promedio);

prom_4 = promedio * promedio * promedio * promedio;
printf("El promedio a la cuarta es:%.2f\n", prom_4);
} else {
printf("El promedio no es mayor a 0\n");
}
}
```

13.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio101.exe'
Ingrese el valor de num1:30
Ingrese el valor de num2:71
El promedio es:50.50
El promedio a la cuarta es:6503775.00
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio101.exe'
Ingrese el valor de num1:4
Ingrese el valor de num2:0
El promedio es:2.00
El promedio a la cuarta es:16.00
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio101.exe'
Ingrese el valor de num1:0
Ingrese el valor de num2:0
El promedio no es mayor a 0
```

14.1. Enunciado del algoritmo

Ejercicio 119

En una floreria, diseñe un algoritmo que lea el radio (real) y la altura (real) de un cilindro. Use la constante simbolica PI = 3.14159. Calcule el area lateral con la expresion aritmetica (2 * PI *

radio * altura) y asignela a area_lateral. Evalúe la expresion logica (radio > 0 AND altura > 0). Muestre por pantalla el area lateral. Adicionalmente, redondee el resultado a 2 decimales. Agregue una validacion para asegurar que los datos sean positivos.

14.2. Análisis del problema

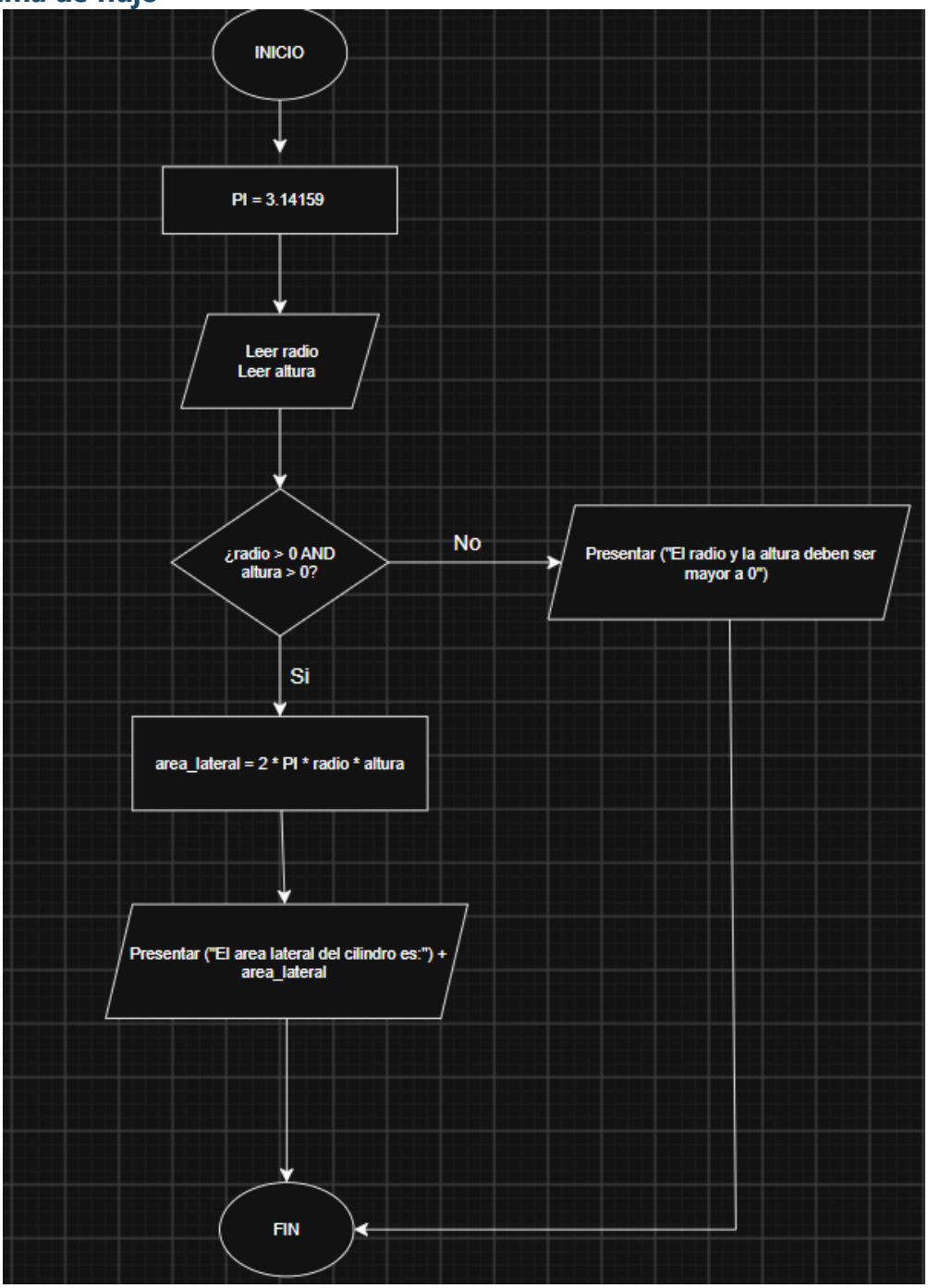
Datos de entrada

1. Valor de radio
2. Valor de la altura

Datos de salida

1. El área lateral (redondeado a 2 decimales)

14.3. Diagrama de flujo



14.4. Pseudocódigo

```
INICIO
//Declarar variables
PI = 3.14159 como constante;
Leer radio como real;
Leer altura como real;
//Proceso
SI radio > 0 AND altura > 0
area_lateral = 2 * PI * radio * altura;
//Salida
Presentar ("El area lateral del cilindro es:") + area_lateral;
Sino
Presentar ("El radio y la altura deben ser mayor a 0");
FIN_SI
FIN
```

14.5. Prueba de escritorio

Prueba	Radio	Altura	Área lateral del cilindro	Mensaje
1	5	12	376.99	Ninguno
2	12	45	3392.92	Ninguno
3	0	0	No se calcula	El radio y la altura deben ser mayor a 0

14.6. Codificación en C

```c
#include <stdio.h>
int main () {
    float PI  = 3.14159, radio = 0.0, altura = 0.0, area_lateral = 0.0;
    printf("Ingrese el radio:");
    scanf("%f", &radio);
    printf("Ingrese la altura:");
    scanf("%f", &altura);
    if (radio > 0 && altura > 0) {
        area_lateral = 2 * PI * radio * altura;
        printf("El area lateral del cilindro es:%.2f\n", area_lateral);
    } else {
        printf("El radio y la altura deben ser mayor a 0\n");
    }

}
```

14.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio119.exe'
Ingrese el radio:5
Ingrese la altura:12
El area lateral del cilindro es:376.99
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio119.exe'
Ingrese el radio:12
Ingrese la altura:45
El area lateral del cilindro es:3392.92
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio119.exe'
Ingrese el radio:0
Ingrese la altura:0
El radio y la altura deben ser mayor a 0
```

15.1. Enunciado del algoritmo

Ejercicio 128

En una gasolinera, diseñe un algoritmo que lea una velocidad en mph (real). Use una constante simbolica FACTOR = 1.60934. Calcule la equivalencia en km/h con la expresion aritmetica (mph * FACTOR) y asignela a kmh. Evalúe la expresion relacional (kmh > 0). Muestre por pantalla 'X mph equivalen a Y km/h'. Ademas del calculo principal, determine el valor maximo de las variables utilizadas. Justifique el tipo de dato simple elegido para cada variable.

15.2. Análisis del problema

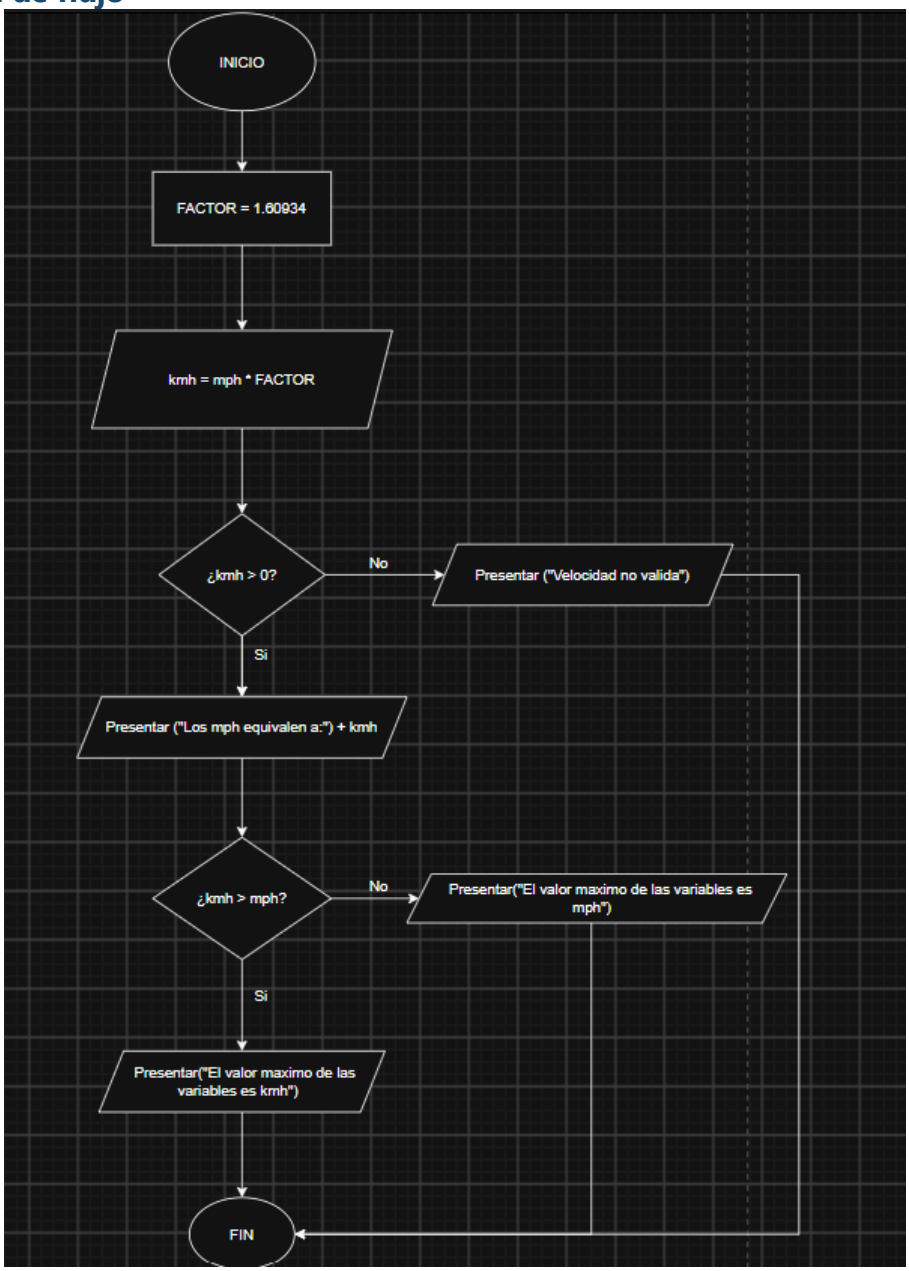
Datos de entrada

1. Valor de la velocidad en mph

Datos de salida

1. Equivalencia de mph en km/h
2. Valor máximo de las variables utilizadas

15.3. Diagrama de flujo



15.4. Pseudocódigo

```
INICIO
//Declarar variables
FACTOR = 1.609340 como constante;
Leer mph como real;
//Entrada
mph = 0.0;
//Proceso
kmh = mph * FACTOR;
SI kmh > 0
//Salida
Presentar ("Los mph equivalen a:") + kmh;
Sino
Presentar ("Velocidad no valida")
FIN_SI
SI kmh > mph
//Salida
Presentar("El valor maximo de las variables es kmh");
Sino
Presentar("El valor maximo de las variables es mph");
FIN_SI
FIN
```

15.5. Prueba de escritorio

Prueba	Minutos ingresados	Tiempo (horas:minutos)	Valor absoluto
1	60	1:00	1
2	120	2:00	2
3	200	3:20	17

15.6. Codificación en C

```c
# include <stdio.h>
int main () {
    float FACTOR = 1.60934, kmh = 0.0, mph = 0.0, maximo = 0.0;
    printf("Ingrese la velocidad en mph: ");
    scanf("%f", &mph);
    kmh = mph * FACTOR;
    if (kmh > 0) {
        printf("Los %.2fmp/h equivalen a: %.2fkm/h\n",mph, kmh);

        if (kmh > mph) {
        maximo = kmh;
    } else {
        maximo = mph;
        }
        printf("El valor maximo de las variables es: %.2f\n", maximo);
        } else {
        printf("El valor maximo de las variables es: %.2f\n", maximo);
        printf("Velocidad no valida\n");
    }
}
```

15.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio128.exe'
Ingrese la velocidad en mph: 30
Los 30.00mp/h equivalen a: 48.28km/h
El valor maximo de las variables es: 48.28
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio128.exe'
Ingrese la velocidad en mph: 189
Los 189.00mp/h equivalen a: 304.17km/h
El valor maximo de las variables es: 304.17
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\.vscode\output> & .\'Ejercicio128.exe'
Ingrese la velocidad en mph: 0
El valor maximo de las variables es: 0.00
Velocidad no valida
```

16.1. Enunciado del algoritmo

Ejercicio 220

En una zapateria, diseñe un algoritmo que lea una cantidad de minutos (entero). Use la constante literal 60. Calcule las horas con (minutos / 60) y los minutos restantes con (minutos % 60). Asigne los resultados a variables enteras horas y min_restantes. Evalúe la expresion logica (minutos >= 0). Muestre por pantalla el tiempo en formato H:MM. Ademas, calcule el valor absoluto de la diferencia entre dos resultados. Identifique explicitamente los datos de entrada y la informacion de salida.

16.2. Análisis del problema

Datos de entrada

1. Cantidad de minutos

Datos de salida

1. Cálculo de horas en formato H:MM
2. Valor absoluto de la diferencia entre dos resultados

16.3. Diagrama de flujo



16.4. Pseudocódigo

```
INICIO
//Declarar variables
Leer minutos como entero;
//Entrada
minutos = 0;
//Proceso
horas = minutos / 60;
min_restantes = minutos % 60;
SI minutos >= 0
//Salida
```

```
Presentar ("El tiempo es") + horas;
Sino
Presentar ("Minutos no validos");
FIN_SI
valor_abs = abs(horas - min_restantes);
//Salida
Presentar ("El valor absoluto es") + valor_abs;
FIN
```

16.5. Prueba de escritorio

Prueba	Minutos ingresados	Tiempo (horas:minutos)	Valor absoluto
1	60	1:00	1
2	120	2:00	2
3	200	3:20	17

16.6. Codificación en C

```c
#include <stdio.h>
#include <stdlib.h>
int main () {
int minutos = 0, horas = 0, valor_abs = 0, min_restantes = 0;

printf("Ingrese minutos:");
scanf("%d", &minutos);

horas = minutos / 60 ;
min_restantes = minutos % 60;

if (minutos >= 0) {
printf("El tiempo es de: %d:%02d\n", horas, min_restantes);
valor_abs = abs(horas - min_restantes);
printf("El valor absoluto es: %d\n", valor_abs);
} else {
printf("Minutos no validos\n");
}

}
```

16.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio220.exe'
Ingrese minutos:60
El tiempo es de: 1:00
El valor absoluto es: 1
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio220.exe'
Ingrese minutos:120
El tiempo es de: 2:00
El valor absoluto es: 2
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio220.exe'
Ingrese minutos:200
El tiempo es de: 3:20
El valor absoluto es: 17
```

17.1. Enunciado del algoritmo

Ejercicio 223

En un hospital, diseñe un algoritmo que lea cinco edades (enteros). Use la constante literal 5 como divisor. Calcule el promedio con la expresion aritmetica ((e1 + e2 + e3 + e4 + e5) / 5) y asignelo a promedio. Evalúe la expresion relacional (promedio > 0). Muestre por pantalla la edad promedio. Ademas del calculo principal, determine si el resultado es positivo o negativo. Asegurese de inicializar todas las variables antes de usarlas.

17.2. Análisis del problema

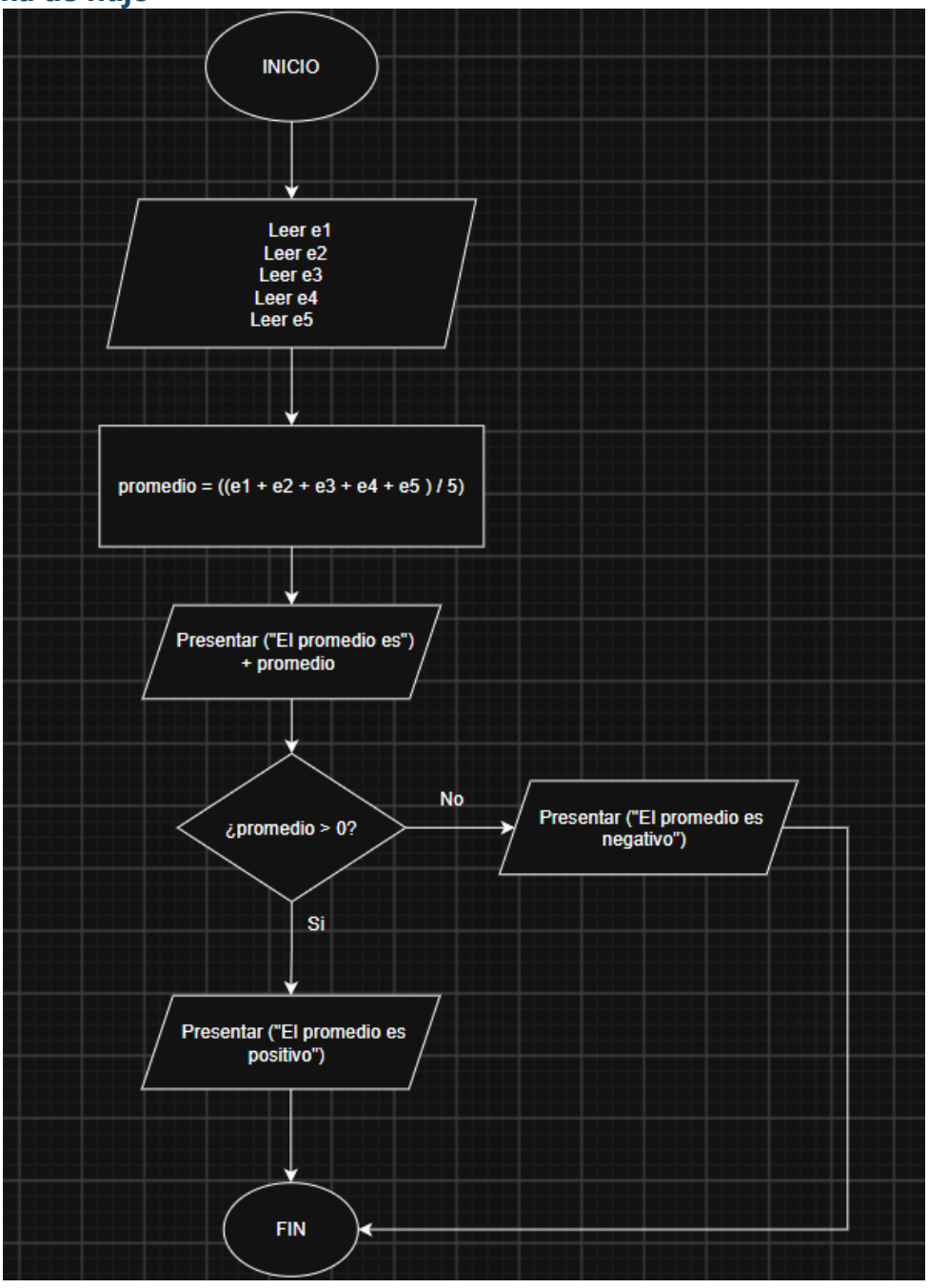
Datos de entrada

1. Valor de las 5 edades (e1, e2, e3, e4, e5)

Datos de salida

1. Edad promedio
2. Si el resultado es positivo o negativo

17.3. Diagrama de flujo



17.4. Pseudocódigo

```
INICIO
//Declarar variables
Leer e1 como entero;
Leer e2 como entero;
Leer e3 como entero;
Leer e4 como entero;
Leer e5 como entero;
//Entrada
e1 = 0;
e2 = 0;
e3 = 0;
e4 = 0;
e5 = 0;
//Proceso
promedio = ((e1 + e2 + e3 + e4 + e5 ) / 5);
//Salida
Presentar ("El promedio es") + promedio;
SI promedio > 0
//Salida
Presentar ("El promedio es positivo");
Sino
Presentar ("El promedio es negativo");
FIN_SI
FIN
```

17.5. Prueba de escritorio

Prueba	e1	e2	e3	e4	e5	Promedio	Mensaje
1	10	11	13	12	11	11.00	El promedio es postivo
2	-10	-23	-32	-24	-18	-21.00	El promedio es negativo

17.6. Codificación en C

```c
#include <stdio.h>
#include <stdlib.h>
int main () {
int e1 = 0, e2 = 0, e3 = 0, e4 = 0, e5 = 0;
float promedio = 0.0;
printf("Ingrese el valor de e1:");
scanf("%d", &e1);
printf("Ingrese el valor de e2:");
scanf("%d", &e2);
printf("Ingrese el valor de e3:");
scanf("%d", &e3);
printf("Ingrese el valor de e4:");
scanf("%d", &e4);
printf("Ingrese el valor de e5:");
scanf("%d", &e5);

promedio = (float)((e1 + e2 + e3 + e4 + e5 ) / 5);
printf("El promedio es:%.2f\n", promedio);

if (promedio > 0) {
printf("El promedio es postivo\n");
} else {
printf("El promedio es negativo\n");
}
}
```

17.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio223.exe'
Ingrese el valor de e1:10
Ingrese el valor de e2:11
Ingrese el valor de e3:13
Ingrese el valor de e4:12
Ingrese el valor de e5:11
El promedio es:11.00
El promedio es postivo
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio223.exe'
Ingrese el valor de e1:-10
Ingrese el valor de e2:-23
Ingrese el valor de e3:-32
Ingrese el valor de e4:-24
Ingrese el valor de e5:-18
El promedio es:-21.00
El promedio es negativo
```

18.1. Enunciado del algoritmo

Ejercicio 246

En una tienda de ropa, construya un algoritmo que lea las bases B y b (reales) y la altura h (real) de un trapecio. Use la constante literal 2. Calcule el area con la expresion aritmetica ((B + b) * h / 2) y asignela a area. Evalúe la expresion logica (B > b AND h > 0). Muestre por pantalla el area del trapecio. Ademas del calculo principal, multiplique el resultado por 1.5 como bono. Incluya una variable auxiliar para llevar un contador de operaciones.

18.2. Análisis del problema

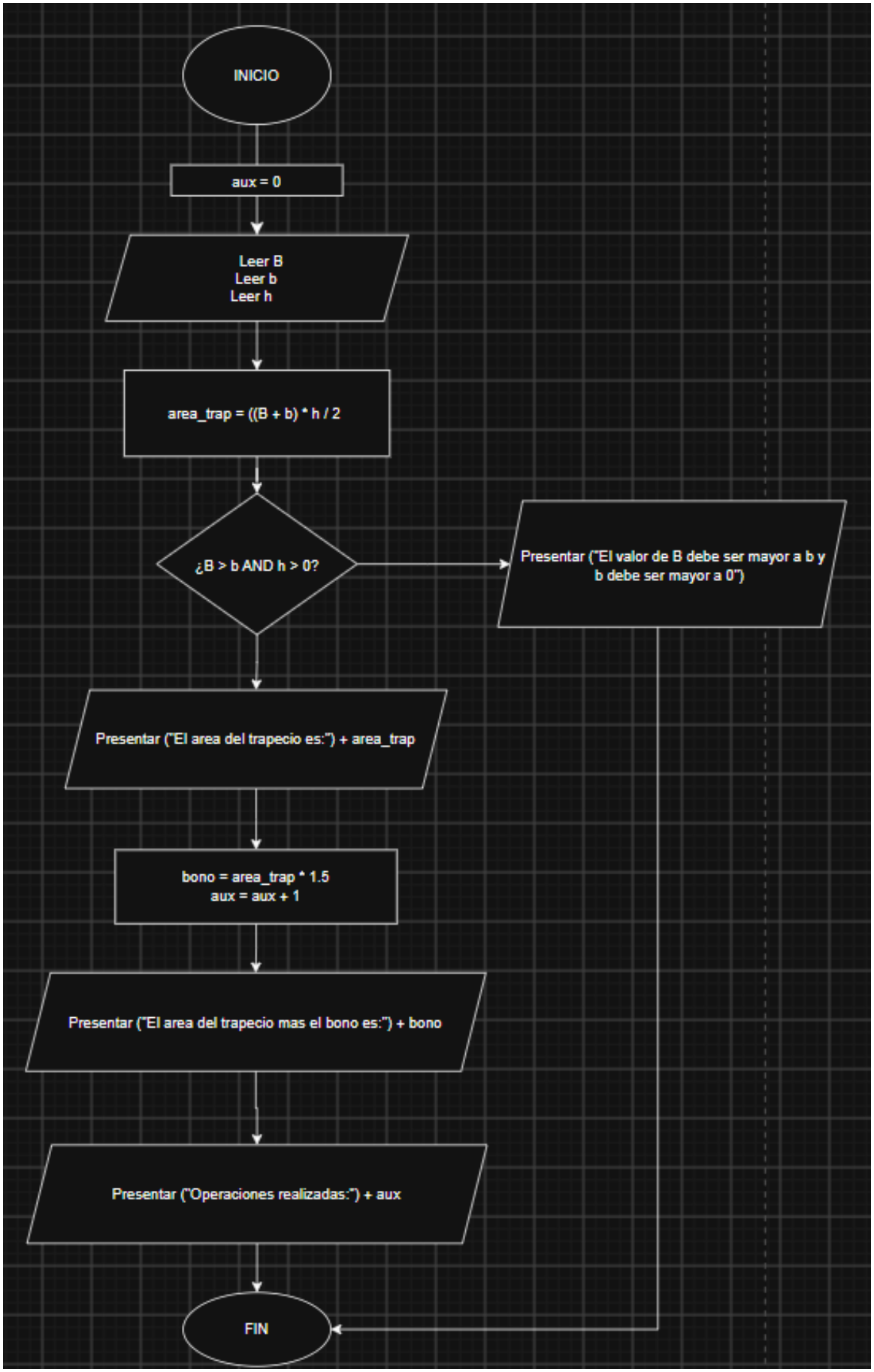
Datos de entrada

1. Valor de B
2. Valor de b
3. Valor de la altura

Datos de salida

1. Área del trapecio
2. Resultado multiplicado por el bono
3. Operaciones realizadas

18.3. Diagrama de flujo



18.4. Pseudocódigo

```
INICIO
//Declarar variables
aux = 0 como auxiliar;
Leer B como real;
Leer b como real;
Leer h como real;
//Entrada
B = 0.0;
b = 0.0;
h = 0.0;
//Proceso
area_trap = ((B + b) * h / 2);
SI B > b AND h > 0
//Salida
Presentar ("El area del trapecio es:") + area_trap;
Sino
Presentar ("El valor de B debe ser mayor a b y b debe ser mayor a 0");
FIN_SI
bono = area_trap * 1.5;
aux = aux + 1;
//Salida
Presentar ("El area del trapecio mas el bono es:") + bono;
//Salida
Presentar ("Operaciones realizadas:") + aux;
FIN
```

18.5. Prueba de escritorio

Prueba	B	b	h	Área del trapecio	Área del trapecio más el bono	Operaciones realizadas	Mensaje
1	25	15	22.6	452.00	678.00	2	Ninguno
2	56	32	19	836.00	1254.00	2	Ninguno
3	13	12	0	No válido	No válido	1	El valor de B debe ser mayor a b y h debe ser mayor a 0

18.6. Codificación en C

```c
#include <stdio.h>
int main () {
int aux = 0;
float B = 0.0, b = 0.0, h = 0.0, area_trap = 0.0, bono = 0.0;
printf("Ingrese el valor de B:");
scanf("%f", &B);

printf("Ingrese el valor de b:");
scanf("%f", &b);

printf("Ingrese el valor de h:");
scanf("%f", &h);

area_trap = ((B + b) * h / 2);
aux = aux + 1;

if(B > b && h > 0) {
printf("El area del trapecio es: %.2f\n", area_trap);
bono = (area_trap * 1.5);
aux = aux + 1;
printf("El area del trapecio mas el bono es: %.2f\n", bono);
} else {
printf("El valor de B debe ser mayor a b y h debe ser mayor a 0\n");
}
printf("Operaciones realizadas: %d\n", aux);
}
```

18.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio246.exe'
Ingrese el valor de B:25
Ingrese el valor de b:15
Ingrese el valor de h:22.6
El area del trapecio es: 452.00
El area del trapecio mas el bono es: 678.00
Operaciones realizadas: 2
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio246.exe'
Ingrese el valor de B:56
Ingrese el valor de b:32
Ingrese el valor de h:19
El area del trapecio es: 836.00
El area del trapecio mas el bono es: 1254.00
Operaciones realizadas: 2
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio246.exe'
Ingrese el valor de B:13
Ingrese el valor de b:12
Ingrese el valor de h:0
El valor de B debe ser mayor a b y h debe ser mayor a 0
Operaciones realizadas: 1
```

19.1. Enunciado del algoritmo

Ejercicio 304

En un gimnasio, cree un algoritmo que solicite el nombre de una persona (cadena), su año de nacimiento (entero) y el año actual (entero). Use una constante simbolica MAYORIA_EDAD = 18. Calcule la edad con la expresion aritmetica (año_actual - año_nacimiento) y asignela a edad. Evalúe la expresion relacional (edad >= MAYORIA_EDAD) y muestre por pantalla 'X tiene Y años y es mayor/menor de edad'. Adicionalmente, calcule la media aritmetica de varios resultados. Use identificadores descriptivos para todas las variables.

19.2. Análisis del problema

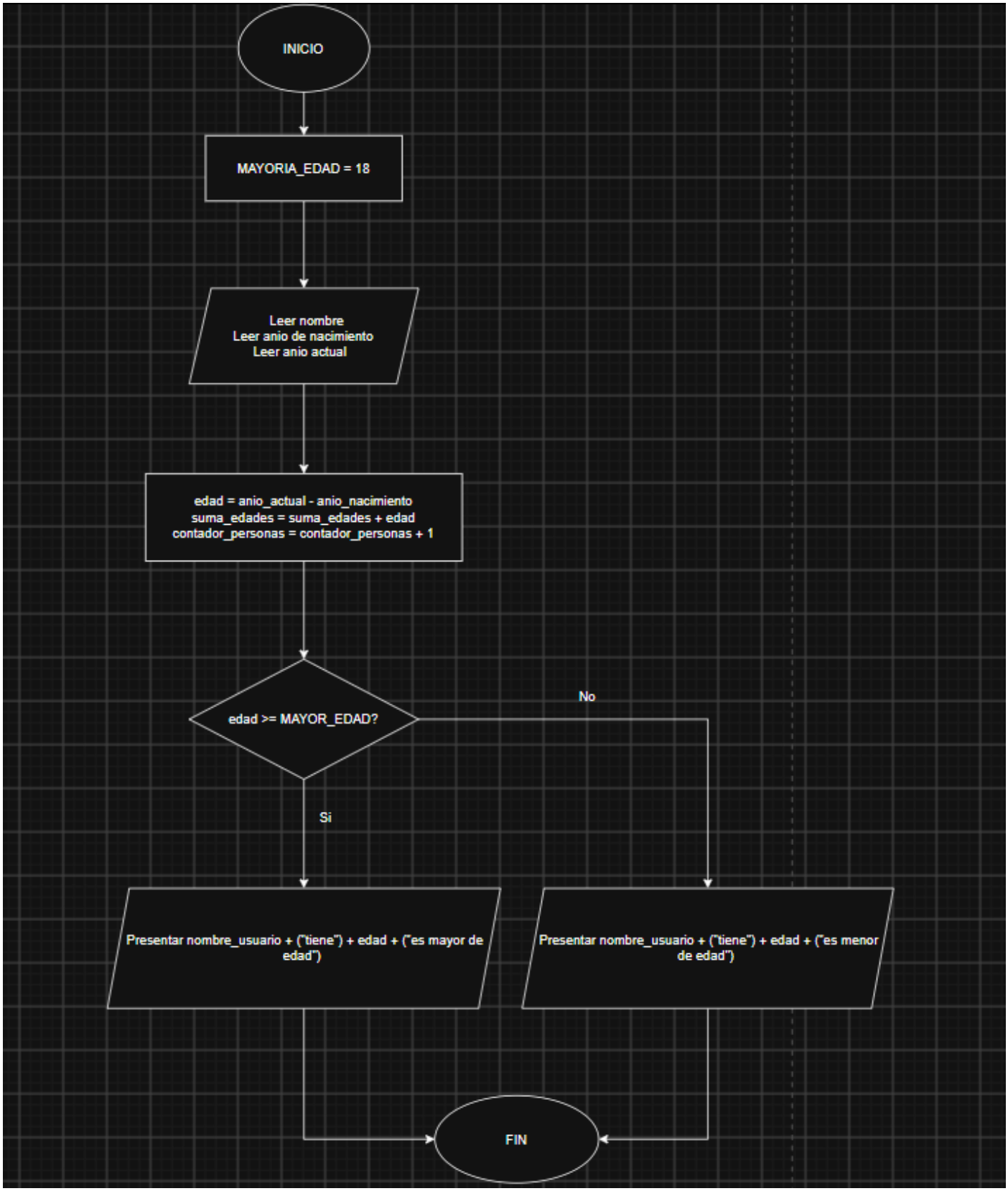
Datos de entrada

1. Nombre
2. Año de nacimiento
3. Año actual

Datos de salida

1. Nombre
2. Edad
3. Si es mayor o menor de edad

19.3. Diagrama de flujo



19.4. Pseudocódigo

```
INICIO
//Declarar variables
MAYORIA_EDAD = 18 como constante;
Leer nombre como cadena;
Leer anio de nacimiento como entero;
Leer anio actual como entero;
```

```
//Entrada
char nombre[30];
anio_nacimiento = 0;
anio_actual = 0;
edad = 0;
//Proceso
edad = anio_actual - anio_nacimiento;
SI edad >= MAYORIA_EDAD ENTONCES;
//Salida
Presentar nombre_usuario + ("tiene") + edad + ("es mayor de edad");
Sino
Presentar nombre_usuario + ("tiene") + edad + ("es menor de edad");
FIN_SI
FIN
```

19.5. Prueba de escritorio

Prueba	Nombre	Año de nacimiento	Año actual	Edad	Resultado
1	Alison	2008	2026	18	Alison tiene 18 años y es mayor de edad
2	Danna	2012	2026	14	Danna tiene 14 años y es menor de edad

19.6. Codificación en C

```c
#include <stdio.h>
int main () {
int MAYORIA_EDAD = 18;
int anio_nacimiento = 0, anio_actual = 0;
int edad = 0;
char nombre[30];

printf ("Ingrese su nombre:");
scanf (" %29[^\n]", nombre);
printf ("Ingrese su anio de nacimiento:");
scanf ("%d", &anio_nacimiento);
printf ("Ingrese anio actual:");
scanf ("%d", &anio_actual);

edad = anio_actual - anio_nacimiento;
// Mayor de edad o no
if (edad >= MAYORIA_EDAD) {
printf("%s tiene %d anios y es mayor de edad\n", nombre, edad);
} else {
  printf("%s tiene %d anios y es menor de edad\n", nombre, edad);
    }

}
```

19.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio304.exe'
Ingrese su nombre:Alison
Ingrese su anio de nacimiento:2008
Ingrese anio actual:2026
Alison tiene 18 anios y es mayor de edad
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio304.exe'
Ingrese su nombre:Danna
Ingrese su anio de nacimiento:2012
Ingrese anio actual:2026
Danna tiene 14 anios y es menor de edad
```

20.1. Enunciado del algoritmo

Ejercicio 312

En una cafeteria, diseñe un algoritmo que lea un caracter por teclado en la variable c. Evalúe las expresiones logicas: (c == 'a' OR c == 'e' OR c == 'i' OR c == 'o' OR c == 'u'). Asigne el resultado booleano a es_vocal_minuscula. Muestre por pantalla el resultado. Justifique el uso del operador OR en lugar de AND. Adicionalmente, calcule el resultado divido entre una constante simbolica. Agregue una variable de estado con el codigo de exito (0) al final.

20.2. Análisis del problema

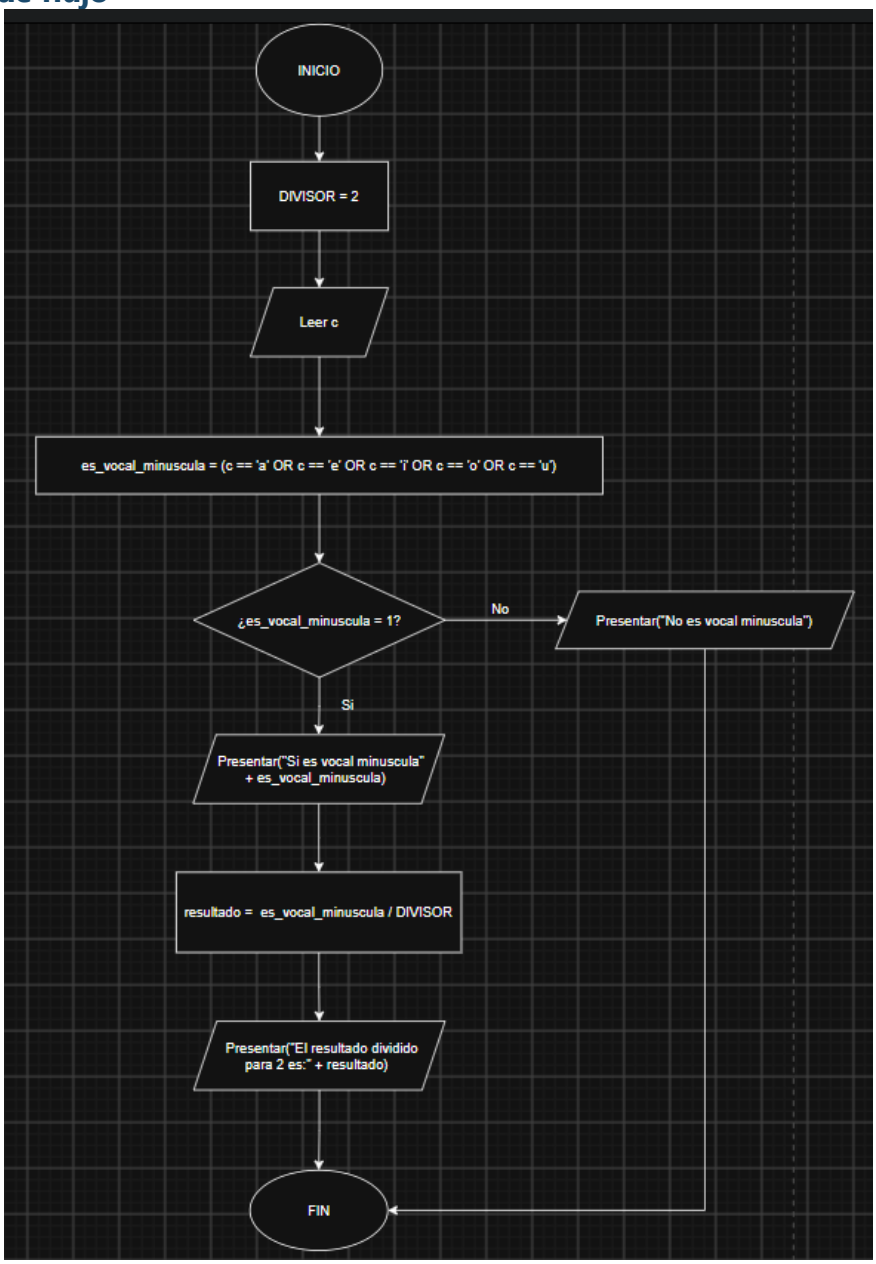
Datos de entrada

1. Carácter (c)

Datos de salida

1. Si es vocal o no
2. Resultado dividido por la constante simbólica

20.3. Diagrama de flujo



20.4. Pseudocódigo

```
INICIO
//Declarar variables
DIVISOR = 2 como constante simbolica;
Leer c como caracter;
//Entrada
c = ' ';
//Proceso
es_vocal_minuscula = (c == 'a' OR c == 'e' OR c == 'i' OR c == 'o' OR c == 'u');
SI es_vocal_minuscula = 1;
//Salida
Presentar("Si es vocal minuscula" + es_vocal_minuscula);
resultado =  es_vocal_minuscula / DIVISOR;
//Salida
Presentar("El resultado dividido para 2 es:" + resultado);
Sino
Presentar("No es vocal minuscula");
FIN
```

20.5. Prueba de escritorio

Prueba	Carácter	Mensaje	Resultado dividido para 2
1	a	'a' sí es vocal minúscula	0.50
2	A	No es vocal minúscula	No válido
3	o	'o' sí es vocal minúscula	0.50

20.6. Codificación en C

```c
#include <stdio.h>
int main () {
    char c = ' ';
    int DIVISOR = 2, es_vocal_minuscula = 0;
    float resultado = 0.0;
    printf("Ingrese un caracter:");
    scanf("%c", &c);
    es_vocal_minuscula = (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u');
    if (es_vocal_minuscula == 1) {
        printf("'%c' si es vocal minuscula\n", c);
        resultado = (float) es_vocal_minuscula / DIVISOR;
        printf("El resultado dividio para %d es:%.2f\n", DIVISOR, resultado);
    } else {
        printf("No es vocal minuscula\n");
    }

}
```

20.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio312.exe'
Ingrese un caracter:a
'a' si es vocal minuscula
El resultado dividio para 2 es:0.50
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio312.exe'
Ingrese un caracter:A
No es vocal minuscula
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio312.exe'
Ingrese un caracter:o
'o' si es vocal minuscula
El resultado dividio para 2 es:0.50
```

21.1. Enunciado del algoritmo

Ejercicio 426

En una escuela, cree un algoritmo que lea tres lados (reales) de un triangulo. Evalúe las expresiones logicas (lado1 + lado2 > lado3) y (lado1 + lado3 > lado2) y (lado2 + lado3 > lado1). Asigne el resultado de combinar las tres con AND a una variable es_triangulo_valido. Muestre por pantalla si los lados forman un triangulo valido. Justifique cada tipo de dato. Ademas, calcule el cuadrado del resultado principal. Muestre una tabla simulada con los datos y resultados al final.

21.2. Análisis del problema

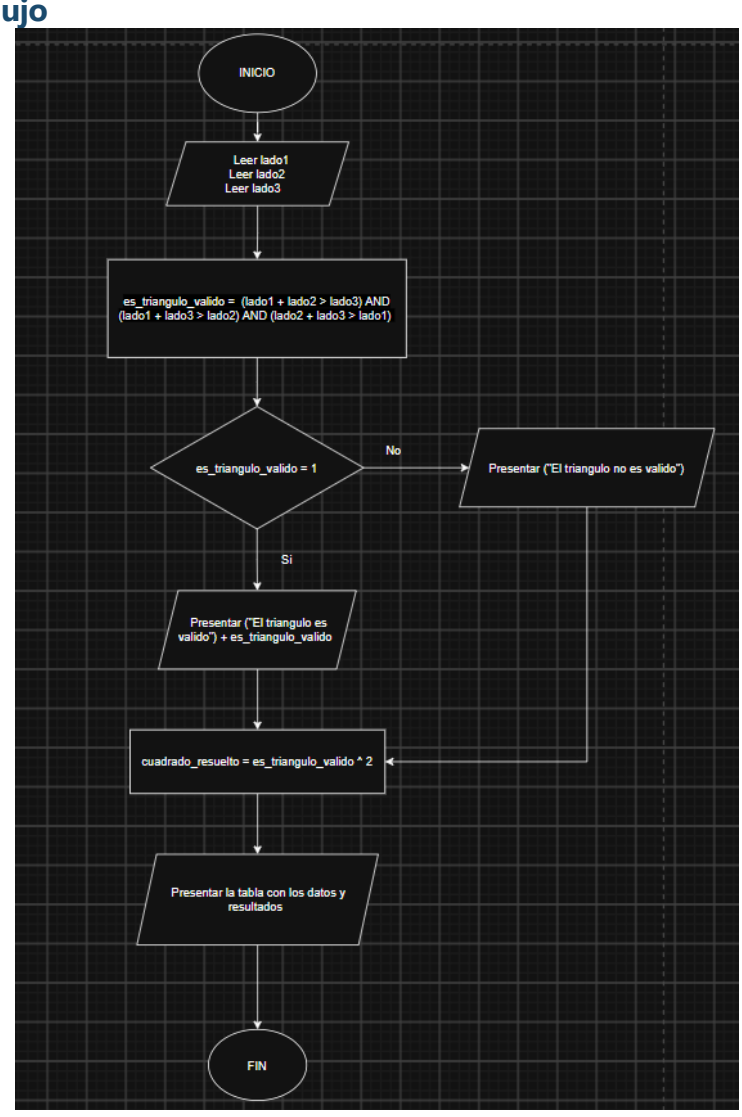
Datos de entrada

1. Valor del lado1
2. Valor del lado2
3. Valor del lado3

Datos de salida

1. Si los lados forman un triangulo
2. Cuadrado del resultado principal
3. Tabla simulada con los resultados

21.3. Diagrama de flujo



21.4. Pseudocódigo

```
INICIO
//Declarar variables
Leer lado1 como real;
Leer lado2 como real;
Leer lado3 como real;
//Entrada
lado1 = 0.0;
lado2 = 0.0;
lado3 = 0.0;
//Proceso
es_triangulo_valido =  (lado1 + lado2 > lado3) AND (lado1 + lado3 > lado2) AND (lado2 + lado3 >
lado1)
SI es_triangulo_valido = 1
//Salida
Presentar ("El triangulo es valido") + es_triangulo_valido;
Sino
Presentar ("El triangulo no es valido");
FIN_SI
cuadrado_resuelto = es_triangulo_valido ^ 2;
//Salida
Presentar la tabla con los datos y resultados;
FIN
```

21.5. Prueba de escritorio

Prueba	lado1	lado2	lado3	Mensaje	triangulo_valido	cuadrado
1	13.00	14.00	15.00	El triángulo es válido	1	1
2	3.00	4.00	5.00	El triángulo es válido	1	1
3	0.00	0.00	0.00	El triángulo no es válido	0	0

21.6. Codificación en C

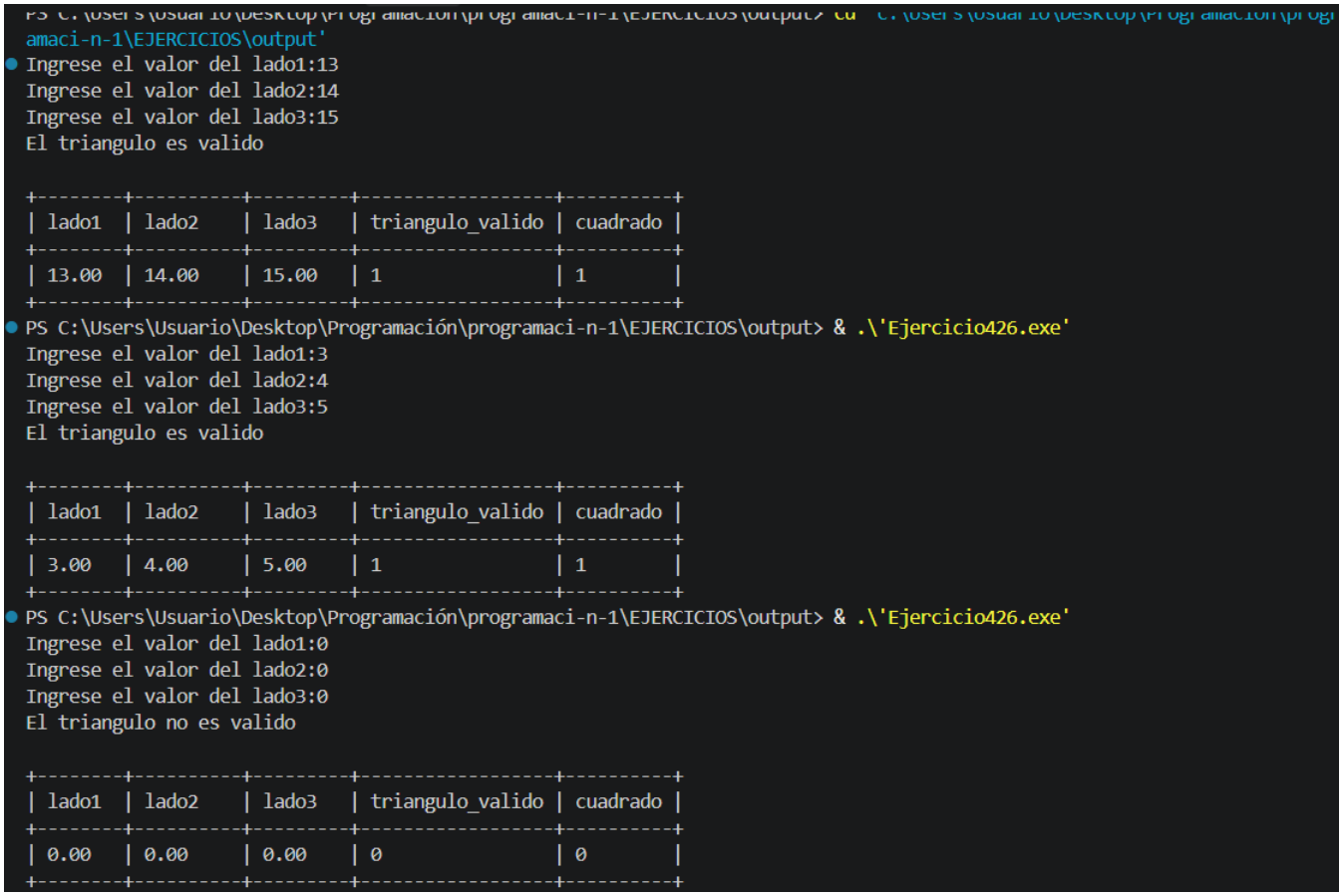
```c
#include <stdio.h>
#include <math.h>
int main () {
    float lado1 = 0.0, lado2 = 0.0, lado3 = 0.0;
    int es_triangulo_valido = 0, cuadrado_resuelto = 0;

    printf("Ingrese el valor del lado1:");
    scanf("%f", &lado1);
    printf("Ingrese el valor del lado2:");
    scanf("%f", &lado2);
    printf("Ingrese el valor del lado3:");
    scanf("%f", &lado3);
    es_triangulo_valido =  (lado1 + lado2 > lado3) && (lado1 + lado3 > lado2) && (lado2 + lado3 > lado1);
    if (es_triangulo_valido == 1) {
        printf("El triangulo es valido\n");

    } else {
        printf("El triangulo no es valido\n");
    }
    cuadrado_resuelto = pow(es_triangulo_valido, 2);
    // Tabla con datos y resultados

    printf("\n+--------+----------+---------+------------------+----------+\n");
    printf("| lado1  | lado2    | lado3   | triangulo_valido | cuadrado |\n");
    printf("+--------+----------+---------+------------------+----------+\n");
    printf("| %-6.2f | %-6.2f   | %-6.2f  | %-16d | %-8d |\n", lado1, lado2, lado3, es_triangulo_valido, cuadrado_resuelto);
    printf("+--------+----------+---------+------------------+----------+\n");
}
```

21.7. Salida del programa



22.1. Enunciado del algoritmo

Ejercicio 435

En una empresa de transporte, cree un algoritmo que lea el numero de revoluciones (entero) y el tiempo en segundos (real). Use la constante simbolica PI = 3.14159. Calcule la velocidad angular con la expresion aritmetica ((revoluciones * 2 * PI) / tiempo) y asignela a velocidad_angular. Evalúe la expresion logica (tiempo > 0). Muestre por pantalla la velocidad angular en rad/s. Ademas, calcule el coseno del angulo resultado (si aplica). Agregue una verificacion para evitar division por cero.

22.2. Análisis del problema

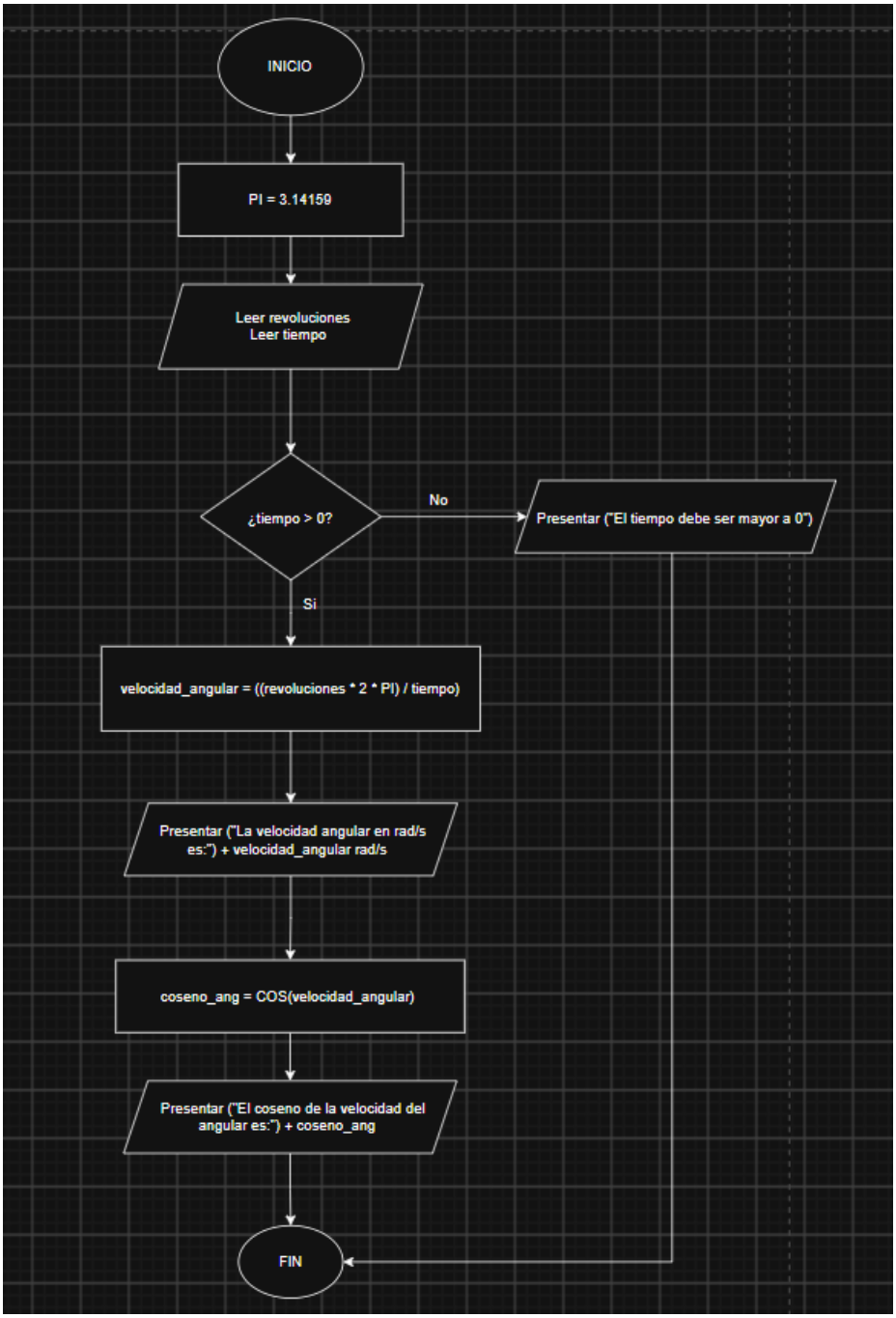
Datos de entrada

1. Número de revoluciones
2. Tiempo en segundos

Datos de salida

1. Velocidad angular en rad/s
2. Coseno del ángulo resultado

22.3. Diagrama de flujo



22.4. Pseudocódigo

```
INICIO
//Declaración de variables
PI = 3.14159 como constante;
Leer revoluciones como entero;
Leer tiempo como real;
//Entrada
revoluciones = 0;
tiempo = 0.0;
//Proceso
SI tiempo > 0
velocidad_angular = ((revoluciones * 2 * PI) / tiempo);
//Salida
Presentar ("La velocidad angular en rad/s es:") + velocidad_angular rad/s;
Sino
Presentar ("El tiempo debe ser mayor a 0");
FIN_SI
```

```
coseno_ang = cos(velocidad_angular);
//Salida
Presentar ("El coseno de la velocidad del angular es:") + coseno_ang;
FIN
```

22.5. Prueba de escritorio

Prueba	Número de revoluciones	Tiempo (s)	Velocidad angular	Coseno de la velocidad angular	Mensaje
1	2	120	0.10 rad/s	0.995	Ninguno
2	60	200	1.88 rad/s	-0.309	Ninguno
3	3	0	No válido	No válido	El tiempo debe ser mayor a 0

22.6. Codificación en C

```c
#include <stdio.h>
#include<math.h>
int main () {
    float PI = 3.14159, tiempo = 0.0, velocidad_angular = 0.0, coseno_ang = 0.0;
    int revoluciones = 0;
printf("Ingrese el numero de revoluciones:");
scanf("%d", &revoluciones);
printf("Ingrese el tiempo en segundos:");
scanf("%f", &tiempo);

if (tiempo > 0) {
velocidad_angular = ((revoluciones * 2 * PI) / tiempo);
printf("La velocidad angular es: %.2f rad/s\n", velocidad_angular);
coseno_ang = cos(velocidad_angular);
printf("El coseno de la velocidad angular es:%.3f\n", coseno_ang);
} else {
printf("El tiempo debe ser mayor a 0\n");

}
}
```

22.7. Salida del programa

```
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio435.exe'
Ingrese el numero de revoluciones:2
Ingrese el tiempo en segundos:120
La velocidad angular es: 0.10 rad/s
El coseno de la velocidad angular es:0.995
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio435.exe'
Ingrese el numero de revoluciones:60
Ingrese el tiempo en segundos:200
La velocidad angular es: 1.88 rad/s
El coseno de la velocidad angular es:-0.309
PS C:\Users\Usuario\Desktop\Programación\programaci-n-1\EJERCICIOS\output> & .\'Ejercicio435.exe'
Ingrese el numero de revoluciones:3
Ingrese el tiempo en segundos:0
El tiempo debe ser mayor a 0
```

3. Conclusiones del algoritmo

- Durante el desarrollo de los ejercicios, las dificultades más frecuentes estuvieron en el momento de programar en lenguaje de C, al comienzo escribía las variables fuera del

printf o del paréntesis, usaba formatos que no correspondían al tipo de dato (por ejemplo: %d para una variable real) y olvidaba símbolos importantes como el & en el scanf. También tuve errores mínimos que no dejaba compilar el código de manera correcta como el punto y coma, no cerrar correctamente las llaves y usar = en vez de == en las condiciones. Estos errores se resolvieron poniendo atención a cada línea del código, entendiendo que tipo de formato usa cada variable, viendo videos y preguntado al docente Ing. José Guamán para despejar dudas.

- Una dificultad inicial fue entender como funcionan correctamente los condicionales en el flujograma, pero con la práctica iba aprendiendo que un rombo equivale a un if de decisión, que el AND es && y OR es || al momento de ejecutar en el lenguaje de C y aprendí a validar antes de calcular como evitar la división por cero.

- Otra dificultad fue saber cómo y cuándo utilizar correctamente las librerías, lo cual fue un reto que se pudo resolver analizando las necesidades del código antes de programar y como resultado se pudo optimizar el código asegurando que cada cabecera incluida cumpliera una función clara según lo pedía el ejercicio.

4. Anexos

https://github.com/alisonnagua/programaci-n.git

5. Autoría

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante

Docente